

*The control of a large force is the same principle as the control of a few men:
it is merely a question of dividing up their numbers.*

— Sun Zi, *The Art of War* (c. 400CE), translated by Lionel Giles (1910)

Our life is frittered away by detail.... Simplify, simplify.

— Henry David Thoreau, *Walden* (1854)

Now, don't ask me what Voom is. I never will know.

But, boy! Let me tell you, it DOES clean up snow!

— Dr. Seuss [Theodor Seuss Geisel], *The Cat in the Hat Comes Back* (1958)

Do the hard jobs first. The easy jobs will take care of themselves.

— attributed to Dale Carnegie

1

Recursion

1.1 Reductions

Reduction is the single most common technique used in designing algorithms. Reducing one problem X to another problem Y means to write an algorithm for X that uses an algorithm for Y as a black box or subroutine. Crucially, the correctness of the resulting algorithm for X cannot depend in any way on *how* the algorithm for Y works. The only thing we can assume is that the black box solves Y correctly. The inner workings of the black box are simply *none of our business*; they're somebody else's problem. It's often best to literally think of the black box as functioning purely by magic.

For example, the peasant multiplication algorithm described in the previous chapter reduces the problem of multiplying two arbitrary positive integers to three simpler problems: addition, mediation (halving), and parity-checking. The algorithm relies on an abstract “positive integer” data type that supports those three operations, but the correctness of the multiplication algorithm does not

depend on the precise data representation (tally marks, clay tokens, Babylonian hexagesimal, quipu, counting rods, Roman numerals, finger positions, augrym stones, gobar numerals, binary, negabinary, Gray code, balanced ternary, phinary, quater-imaginary, . . .), or on the precise implementations of those operations. Of course, the *running time* of the multiplication algorithm depends on the *running time* of the addition, mediation, and parity operations, but that's a separate issue from *correctness*. Most importantly, we can create a more efficient multiplication algorithm just by switching to a more efficient number representation (from tally marks to place-value notation, for example).

Similarly, the Huntington-Hill algorithm reduces the problem of apportioning Congress to the problem of maintaining a priority queue that supports the operations INSERT and EXTRACTMAX. The abstract data type “priority queue” is a black box; the correctness of the apportionment algorithm does not depend on any specific priority queue data structure. Of course, the *running time* of the apportionment algorithm depends on the *running time* of the INSERT and EXTRACTMAX algorithms, but that's a separate issue from the *correctness* of the algorithm. The beauty of the reduction is that we can create a more efficient apportionment algorithm by simply swapping in a new priority queue data structure. Moreover, the designer of that data structure does not need to know or care that it will be used to apportion Congress.

When we design algorithms, we may not know exactly how the basic building blocks we use are implemented, or how our algorithms might be used as building blocks to solve even bigger problems. That ignorance is uncomfortable for many beginners, but it is both unavoidable and extremely useful. Even when you do know precisely how your components work, it is often *extremely* helpful to pretend that you don't.

1.2 Simplify and Delegate

Recursion is a particularly powerful kind of reduction, which can be described loosely as follows:

- If the given instance of the problem can be solved directly, solve it directly.
- Otherwise, reduce it to one or more *simpler instances of the same problem*.

If the self-reference is confusing, it may be helpful to imagine that someone else is going to solve the simpler problems, just as you would assume for other types of reductions. I like to call that someone else the ***Recursion Fairy***. Your *only* task is to *simplify* the original problem, or to solve it directly when simplification is either unnecessary or impossible; the Recursion Fairy will solve all the simpler subproblems for you, using Methods That Are None Of Your Business So *Butt*

Out.¹ Mathematically sophisticated readers might recognize the Recursion Fairy by its more formal name: the *Induction Hypothesis*.

There is one mild technical condition that must be satisfied in order for any recursive method to work correctly: There must be no infinite sequence of reductions to simpler and simpler instances. Eventually, the recursive reductions must lead to an elementary *base case* that can be solved by some other method; otherwise, the recursive algorithm will loop forever. The most common way to satisfy this condition is to reduce to one or more *smaller* instances of the same problem. For example, if the original input is a skreeble with n glurps, the input to each recursive call should be a skreeble with strictly less than n glurps. Of course this is impossible if the skreeble has no glurps at all—You can’t have negative glurps; that would be silly!—so in that case we must grindbloss the skreeble using some other method.

We’ve already seen one instance of this pattern in the peasant multiplication algorithm, which is based directly on the following recursive identity.

$$x \cdot y = \begin{cases} 0 & \text{if } x = 0 \\ \lfloor x/2 \rfloor \cdot (y + y) & \text{if } x \text{ is even} \\ \lfloor x/2 \rfloor \cdot (y + y) + y & \text{if } x \text{ is odd} \end{cases}$$

The same recurrence can be expressed algorithmically as follows:

```

PEASANTMULTIPLY( $x, y$ ):
  if  $x = 0$ 
    return 0
  else
     $x' \leftarrow \lfloor x/2 \rfloor$ 
     $y' \leftarrow y + y$ 
     $prod \leftarrow$  PEASANTMULTIPLY( $x', y'$ )  <<Recurse!>>
    if  $x$  is odd
       $prod \leftarrow prod + y$ 
    return  $prod$ 

```

A lazy Egyptian scribe could execute this algorithm by computing x' and y' , asking a more junior scribe to multiply x' and y' , and then possibly adding y to the junior scribe’s response. The junior scribe’s problem is simpler because $x' < x$, and repeatedly decreasing a positive integer eventually leads to 0. How the junior scribe actually computes $x' \cdot y'$ is none of the senior scribe’s business (and it’s none of your business, either).

¹When I was an undergraduate, I attributed recursion to “elves” instead of the Recursion Fairy, referring to the Brothers Grimm story about an old shoemaker who leaves his work unfinished when he goes to bed, only to discover upon waking that elves (“Wichtelmänner”) have finished everything overnight. Someone more entheogenically experienced than I might recognize these Rekursionswichtelmänner as Terence McKenna’s “self-transforming machine elves”.

1.3 Tower of Hanoi

The Tower of Hanoi puzzle was first published—as an actual physical puzzle!—by the French teacher and recreational mathematician Édouard Lucas in 1883,² under the pseudonym “N. Claus (de Siam)” (an anagram of “Lucas d’Amiens”). The following year, Henri de Parville described the puzzle with the following remarkable story:³

In the great temple at Benares⁴... beneath the dome which marks the centre of the world, rests a brass plate in which are fixed three diamond needles, each a cubit high and as thick as the body of a bee. On one of these needles, at the creation, God placed sixty-four discs of pure gold, the largest disc resting on the brass plate, and the others getting smaller and smaller up to the top one. This is the Tower of Bramah. Day and night unceasingly the priests transfer the discs from one diamond needle to another according to the fixed and immutable laws of Bramah, which require that the priest on duty must not move more than one disc at a time and that he must place this disc on a needle so that there is no smaller disc below it. When the sixty-four discs shall have been thus transferred from the needle on which at the creation God placed them to one of the other needles, tower, temple, and Brahmins alike will crumble into dust, and with a thunderclap the world will vanish.

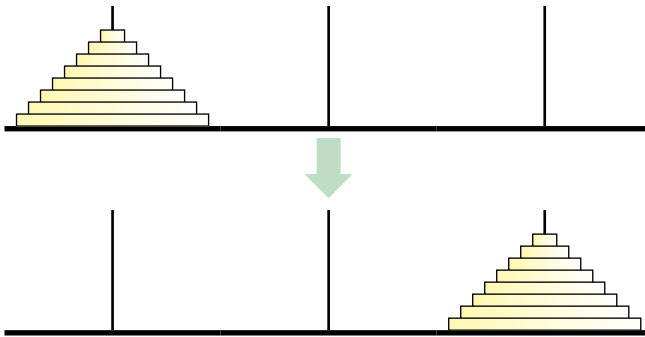


Figure 1.1. The (8-disk) Tower of Hanoi puzzle

Of course, as good computer scientists, our first instinct on reading this story is to substitute the variable n for the hardwired constant 64. And because most physical instances of the puzzle are made of wood instead of diamonds and gold, I will call the three possible locations for the disks “pegs” instead of

²Lucas later claimed to have invented the puzzle in 1876.

³This English translation is taken from W. W. Rouse Ball’s 1892 book *Mathematical Recreations and Essays*.

⁴The “great temple at Benares” is almost certainly the Kashi Vishvanath Temple in Varanasi, Uttar Pradesh, India, located approximately 2400km west-north-west of Hà Nội, Việt Nam, where the fictional N. Claus supposedly resided. Coincidentally, the French Army invaded Hanoi in 1883, the same year Lucas released his puzzle, ultimately leading to its establishment as the capital of French Indochina.

“needles”. How can we move a tower of n disks from one peg to another, using a third spare peg as an occasional placeholder, without ever placing a disk on top of a smaller disk?

As N. Claus (de Siam) pointed out in the pamphlet included with his puzzle, the secret to solving this puzzle is to think recursively. Instead of trying to solve the entire puzzle at once, let’s concentrate on moving just the largest disk. We can’t move it at the beginning, because all the other disks are in the way. So first we have to move those $n - 1$ smaller disks to the spare peg. Once that’s done, we can move the largest disk directly to its destination. Finally, to finish the puzzle, we have to move the $n - 1$ smaller disks from the spare peg to their destination.

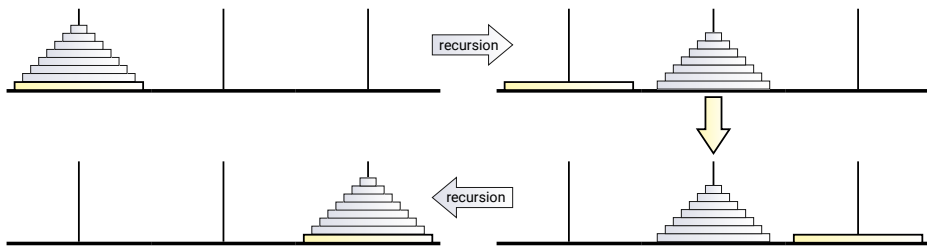


Figure 1.2. The Tower of Hanoi algorithm; ignore everything but the bottom disk.

So now all we have to figure out is how to—

NO!! STOP!!

That’s it! We’re done! We’ve successfully reduced the n -disk Tower of Hanoi problem to two instances of the $(n - 1)$ -disk Tower of Hanoi problem, which we can gleefully hand off to the Recursion Fairy—or to carry Lucas’s metaphor further, to the junior monks at the temple. *Our* job is finished. If we didn’t trust the junior monks, we wouldn’t have hired them; let them do their job in peace.

Our reduction does make one subtle but extremely important assumption: *There is a largest disk*. Our recursive algorithm works for any *positive* number of disks, but it breaks down when $n = 0$. We must handle that case using a different method. Fortunately, the monks at Benares, being good Buddhists, are quite adept at moving zero disks from one peg to another in no time at all, by doing nothing.



Figure 1.3. The vacuous base case for the Tower of Hanoi algorithm. There is no spoon.

It may be tempting to think about how all those smaller disks move around—or more generally, what happens when the recursion is unrolled—but really, don't do it. For most recursive algorithms, unrolling the recursion is neither necessary nor helpful. Our *only* task is to reduce the problem instance we're given to one or more simpler instances, or to solve the problem directly if such a reduction is impossible. Our recursive Tower of Hanoi algorithm is trivially correct when $n = 0$. For any $n \geq 1$, the Recursion Fairy correctly moves the top $n - 1$ disks (more formally, the Inductive Hypothesis implies that our recursive algorithm correctly moves the top $n - 1$ disks) so our algorithm is correct.

The recursive Hanoi algorithm is expressed in pseudocode in Figure 1.4. The algorithm moves a stack of n disks from a source peg (*src*) to a destination peg (*dst*) using a third temporary peg (*tmp*) as a placeholder. Notice that the algorithm correctly does nothing at all when $n = 0$.

<pre>HANOI(<i>n, src, dst, tmp</i>): if <i>n</i> > 0 HANOI(<i>n - 1, src, tmp, dst</i>) <<Recurse!>> move disk <i>n</i> from <i>src</i> to <i>dst</i> HANOI(<i>n - 1, tmp, dst, src</i>) <<Recurse!>></pre>
--

Figure 1.4. A recursive algorithm to solve the Tower of Hanoi

Let $T(n)$ denote the number of moves required to transfer n disks—the running time of our algorithm. Our vacuous base case implies that $T(0) = 0$, and the more general recursive algorithm implies that $T(n) = 2T(n - 1) + 1$ for any $n \geq 1$. By writing out the first several values of $T(n)$, we can easily guess that $T(n) = 2^n - 1$; a straightforward induction proof implies that this guess is correct. In particular, moving a tower of 64 disks requires $2^{64} - 1 = 18,446,744,073,709,551,615$ individual moves. Thus, even at the impressive rate of one move per second, the monks at Benares will be at work for approximately 585 billion years (“plus de *cinq milliards de siècles*”) before tower, temple, and Brahmins alike will crumble into dust, and with a thunderclap the world will vanish.

1.4 Mergesort

Mergesort is one of the earliest algorithms designed for general-purpose stored-program computers. The algorithm was developed by John von Neumann in 1945, and described in detail in a publication with Herman Goldstine in 1947, as one of the first non-numerical programs for the EDVAC.⁵

⁵Goldstine and von Neumann actually described an non-recursive variant now usually called bottom-up mergesort. At the time, large data sets were sorted by special-purpose machines—almost all built by IBM—that manipulated punched cards using variants of binary radix sort. Von

1. Divide the input array into two subarrays of roughly equal size.
2. Recursively mergesort each of the subarrays.
3. Merge the newly-sorted subarrays into a single sorted array.

Input:	S	O	R	T	I	N	G	E	X	A	M	P	L	
Divide:	S	O	R	T	I	N		G	E	X	A	M	P	L
Recurse Left:	I	N	O	R	S	T		G	E	X	A	M	P	L
Recurse Right:	I	N	O	R	S	T		A	E	G	L	M	P	X
Merge:	A	E	G	I	L	M	N	O	P	R	S	T	X	

Figure 1.5. A mergesort example.

The first step is completely trivial—just divide the array size by two—and we can delegate the second step to the Recursion Fairy. All the real work is done in the final merge step. A complete description of the algorithm is given in Figure 1.6; to keep the recursive structure clear, I’ve extracted the merge step into an independent subroutine. The merge algorithm is also recursive—identify the first element of the output array, and then recursively merge the rest of the input arrays.

```

MERGESORT( $A[1..n]$ ):
  if  $n > 1$ 
     $m \leftarrow \lfloor n/2 \rfloor$ 
    MERGESORT( $A[1..m]$ )    «Recurse!»
    MERGESORT( $A[m+1..n]$ ) «Recurse!»
    MERGE( $A[1..n], m$ )

```

```

MERGE( $A[1..n], m$ ):
   $i \leftarrow 1$ ;  $j \leftarrow m + 1$ 
  for  $k \leftarrow 1$  to  $n$ 
    if  $j > n$ 
       $B[k] \leftarrow A[i]$ ;  $i \leftarrow i + 1$ 
    else if  $i > m$ 
       $B[k] \leftarrow A[j]$ ;  $j \leftarrow j + 1$ 
    else if  $A[i] < A[j]$ 
       $B[k] \leftarrow A[i]$ ;  $i \leftarrow i + 1$ 
    else
       $B[k] \leftarrow A[j]$ ;  $j \leftarrow j + 1$ 
  for  $k \leftarrow 1$  to  $n$ 
     $A[k] \leftarrow B[k]$ 

```

Figure 1.6. Mergesort

Correctness

To prove that this algorithm is correct, we apply our old friend induction twice, first to the MERGE subroutine then to the top-level MERGESORT algorithm.

Lemma 1.1. *MERGE correctly merges the subarrays $A[1..m]$ and $A[m+1..n]$, assuming those subarrays are sorted in the input.*

Neumann argued (successfully!) that because the EDVAC could sort faster than IBM’s dedicated sorters, “without human intervention or need for additional equipment”, the EDVAC was an “all purpose” machine, and special-purpose sorting machines were no longer necessary.

Proof: Let $A[1..n]$ be any array and m any integer such that the subarrays $A[1..m]$ and $A[m+1..n]$ are sorted. We prove that for all k from 0 to n , the last $n - k - 1$ iterations of the main loop correctly merge $A[i..m]$ and $A[j..n]$ into $B[k..n]$. The proof proceeds by induction on $n - k + 1$, the number of elements remaining to be merged.

If $k > n$, the algorithm correctly merges the two empty subarrays by doing absolutely nothing. (This is the base case of the inductive proof.) Otherwise, there are four cases to consider for the k th iteration of the main loop.

- If $j > n$, then subarray $A[j..n]$ is empty, so $\min(A[i..m] \cup A[j..n]) = A[i]$.
- If $i > m$, then subarray $A[i..m]$ is empty, so $\min(A[i..m] \cup A[j..n]) = A[j]$.
- Otherwise, if $A[i] < A[j]$, then $\min(A[i..m] \cup A[j..n]) = A[i]$.
- Otherwise, we must have $A[i] \geq A[j]$, and $\min(A[i..m] \cup A[j..n]) = A[j]$.

In all four cases, $B[k]$ is correctly assigned the smallest element of $A[i..m] \cup A[j..n]$. In the two cases with the assignment $B[k] \leftarrow A[i]$, the Recursion Fairy correctly merges—sorry, I mean the Induction Hypothesis implies that the last $n - k$ iterations of the main loop correctly merge $A[i+1..m]$ and $A[j..n]$ into $B[k+1..n]$. Similarly, in the other two cases, the Recursion Fairy also correctly merges the rest of the subarrays. $\square$

Theorem 1.2. *MERGESORT correctly sorts any input array $A[1..n]$.*

Proof: We prove the theorem by induction on n . If $n \leq 1$, the algorithm correctly does nothing. Otherwise, the Recursion Fairy correctly sorts—sorry, I mean the induction hypothesis implies that our algorithm correctly sorts the two smaller subarrays $A[1..m]$ and $A[m+1..n]$, after which they are correctly MERGED into a single sorted array (by Lemma 1.1). $\square$

Analysis

Because the MERGESORT algorithm is recursive, its running time is naturally expressed as a recurrence. MERGE clearly takes $O(n)$ time, because it's a simple for-loop with constant work per iteration. We immediately obtain the following recurrence for MERGESORT:

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n).$$

As in most divide-and-conquer recurrences, we can safely strip out the floors and ceilings (using a technique called *domain transformations* described later in this chapter), giving us the simpler recurrence $T(n) = 2T(n/2) + O(n)$. The “all levels equal” case of the recursion tree method (also described later in this chapter) immediately implies the closed-form solution $T(n) = O(n \log n)$. Even if you are not (yet) familiar with recursion trees, you can verify the solution $T(n) = O(n \log n)$ by induction.

1.5 Quicksort

Quicksort is another recursive sorting algorithm, discovered by Tony Hoare in 1959 and first published in 1961. In this algorithm, the hard work is splitting the array into smaller subarrays *before* recursion, so that merging the sorted subarrays is trivial.

1. Choose a *pivot* element from the array.
2. Partition the array into three subarrays containing the elements smaller than the pivot, the pivot element itself, and the elements larger than the pivot.
3. Recursively quicksort the first and last subarrays.

Input:	S	O	R	T	I	N	G	E	X	A	M	P	L
Choose a pivot:	S	O	R	T	I	N	G	E	X	A	M	P	L
Partition:	A	G	O	E	I	N	L	M	P	T	X	S	R
Recurse Left:	A	E	G	I	L	M	N	O	P	T	X	S	R
Recurse Right:	A	E	G	I	L	M	N	O	P	R	S	T	X

Figure 1.7. A quicksort example.

More detailed pseudocode is given in Figure 1.8. In the PARTITION subroutine, the input parameter p is the index of the pivot element in the unsorted array; the subroutine partitions the array and returns the new index of the pivot element. There are many different efficient partitioning algorithms; the one I'm presenting here is attributed to Nico Lomuto.⁶ The variable ℓ counts the number of items in the array that are ℓ ess than the pivot element.

<pre> QUICKSORT($A[1..n]$): if ($n > 1$) Choose a pivot element $A[p]$ $r \leftarrow \text{PARTITION}(A, p)$ QUICKSORT($A[1..r-1]$) <i>«Recurse!»</i> QUICKSORT($A[r+1..n]$) <i>«Recurse!»</i> </pre>	<pre> PARTITION($A[1..n], p$): swap $A[p] \leftrightarrow A[n]$ $\ell \leftarrow 0$ <i>«#items < pivot»</i> for $i \leftarrow 1$ to $n-1$ if $A[i] < A[n]$ $\ell \leftarrow \ell + 1$ swap $A[\ell] \leftrightarrow A[i]$ swap $A[n] \leftrightarrow A[\ell + 1]$ return $\ell + 1$ </pre>
--	---

Figure 1.8. Quicksort

Correctness

Just like mergesort, proving that QUICKSORT is correct requires two separate induction proofs: **one to prove that PARTITION correctly partitions the array, and**

⁶Hoare proposed a more complicated “two-way” partitioning algorithm that has some practical advantages over Lomuto’s algorithm. On the other hand, Hoare’s partitioning algorithm is one of the places off-by-one errors go to die.

the other to prove that **QUICKSORT correctly sorts assuming PARTITION is correct.** To prove PARTITION is correct, we need to prove the following loop invariant: **At the end of each iteration of the main loop, everything in the subarray $A[1..l]$ is less than $A[n]$, and nothing in the subarray $A[l+1..i]$ is less than $A[n]$.** I'll leave the remaining straightforward but tedious details as exercises for the reader.

Analysis

The analysis of quicksort is also similar to that of mergesort. PARTITION clearly runs in $O(n)$ time, because it's a simple for-loop with constant work per iteration. For QUICKSORT, we get a **recurrence that depends on r** , the *rank* of the chosen pivot element:

$$T(n) = T(r-1) + T(n-r) + O(n)$$

If we could somehow always magically choose the pivot to be the *median* element of the array A , we would have $r = \lceil n/2 \rceil$, the two subproblems would be as close to the same size as possible, the recurrence would become

$$T(n) = T(\lceil n/2 \rceil - 1) + T(\lfloor n/2 \rfloor) + O(n) \leq 2T(n/2) + O(n),$$

and we'd have **$T(n) = O(n \log n)$** using either the recursion tree method or the even simpler “Oh yeah, we already solved that recurrence for mergesort” method.

In fact, as we will see later in this chapter, we *can* actually locate the **median element in an unsorted array in linear time**, but the algorithm is fairly complicated, and the hidden constant in the $O(\cdot)$ notation is large enough to **make the resulting sorting algorithm impractical**. In practice, most programmers settle for something simple, like choosing the first or last element of the array. In this case, r can take any value between 1 and n , so we have

$$T(n) = \max_{1 \leq r \leq n} (T(r-1) + T(n-r) + O(n)).$$

In the worst case, the two subproblems are completely unbalanced—either $r = 1$ or $r = n$ —and the recurrence becomes $T(n) \leq T(n-1) + O(n)$. The solution is **$T(n) = O(n^2)$** .

Another common heuristic is called “median of three”—choose three elements (usually at the **beginning, middle, and end of the array**), and take the median of those three elements as the pivot. Although this heuristic is **somewhat more efficient in practice** than just choosing one element, especially when the array is already (nearly) sorted, we can still have $r = 2$ or $r = n-1$ in the worst case. With the median-of-three heuristic, the recurrence becomes **$T(n) \leq T(1) + T(n-2) + O(n)$** , whose solution is still $T(n) = O(n^2)$.

Intuitively, the pivot element should “usually” fall somewhere in the middle of the array, say with rank between $n/10$ and $9n/10$. This observation suggests that the “average-case” running time should be $O(n \log n)$. Although this intuition can be formalized, the most common formalization makes the completely unrealistic assumption that all permutations of the input array are equally likely. Real world data *may* be random, but it is not random in any way that we can predict in advance, and it is *certainly* not uniform!⁷

Occasionally people also consider “best case” running time for some reason. We won’t.

1.6 The Pattern

Both mergesort and quicksort follow a general three-step pattern called **divide and conquer**:

1. **Divide** the given instance of the problem into several *independent smaller* instances of *exactly* the same problem.
2. **Delegate** each smaller instance to the Recursion Fairy.
3. **Combine** the solutions for the smaller instances into the final solution for the given instance.

If the size of any instance **falls below some constant threshold**, we abandon recursion and **solve the problem directly, by brute force, in constant time**.

Proving a divide-and-conquer algorithm correct almost always requires induction. Analyzing the running time requires setting up and solving a recurrence, which usually (but unfortunately not always!) can be solved using recursion trees.

1.7 Recursion Trees

So what are these “recursion trees” I keep talking about? Recursion trees are a simple, general, pictorial tool for solving divide-and-conquer recurrences. A recursion tree is a rooted tree with one node for each recursive subproblem. The *value* of each node is the amount of time spent on the corresponding subproblem *excluding* recursive calls. Thus, the overall running time of the algorithm is the sum of the values of all nodes in the tree.

To make this idea more concrete, imagine a divide-and-conquer algorithm that spends $O(f(n))$ time on non-recursive work, and then makes r recursive

⁷On the other hand, if we choose the pivot index p uniformly at random, then QUICKSORT runs in $O(n \log n)$ time *with high probability*, for every possible input array. The key difference is that the randomness is controlled by our algorithm, not by the All-Powerful Malicious Adversary who gives us input data after reading our code. The analysis of randomized quicksort is unfortunately outside the scope of this book, but you can find relevant lecture notes at <http://algorithms.wtf/>.

calls, each on a problem of size n/c . Up to constant factors (which we can hide in the $O()$ notation), the running time of this algorithm is governed by the recurrence

$$T(n) = r T(n/c) + f(n).$$

The root of the recursion tree for $T(n)$ has value $f(n)$ and r children, each of which is the root of a (recursively defined) recursion tree for $T(n/c)$. Equivalently, a recursion tree is a complete r -ary tree where each node at depth d contains the value $f(n/c^d)$. (Feel free to assume that n is an integer power of c , so that n/c^d is always an integer, although in fact this doesn't matter.)

In practice, I recommend drawing out the first two or three levels of the tree, as in Figure 1.9.

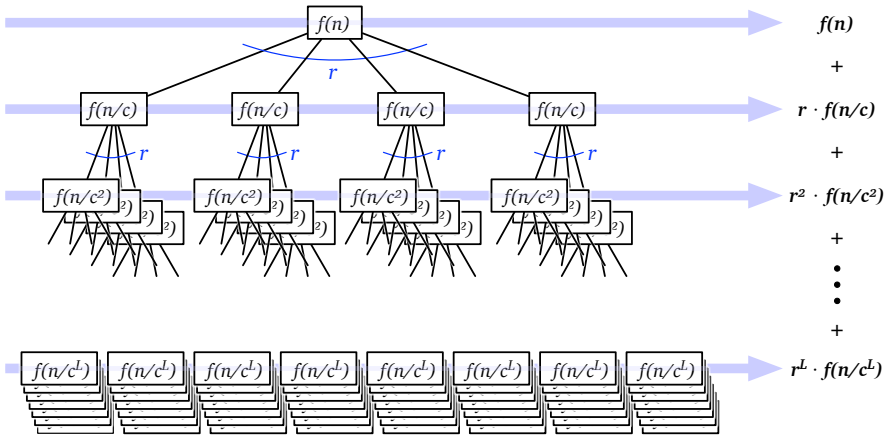


Figure 1.9. A recursion tree for the recurrence $T(n) = r T(n/c) + f(n)$

The leaves of the recursion tree correspond to the base case(s) of the recurrence. Because we're only looking for asymptotic bounds, the precise base case doesn't actually matter; we can safely assume $T(n) = 1$ for all $n \leq n_0$, where n_0 is an arbitrary positive constant. In particular, we can choose whatever value of n_0 is most convenient for our analysis. For this example, I'll choose $n_0 = 1$.

Now $T(n)$ is the sum of all values in the recursion tree; we can evaluate this sum by considering the tree level-by-level. For each integer i , the i th level of the tree has exactly r^i nodes, each with value $f(n/c^i)$. Thus,

$$T(n) = \sum_{i=0}^L r^i \cdot f(n/c^i) \quad (\Sigma)$$

where L is the depth of the tree. Our base case $n_0 = 1$ immediately implies $L = \log_c n$, because $n/c^L = n_0 = 1$. It follows that the number of leaves in

the recursion tree is exactly $r^L = r^{\log_c n} = n^{\log_c r}$. Thus, the last term in the level-by-level sum (Σ) is $n^{\log_c r} \cdot f(1) = O(n^{\log_c r})$, because $f(1) = O(1)$.

There are three common cases where the level-by-level series (Σ) is especially easy to evaluate:

- **Decreasing:** If the series *decays exponentially*—every term is a constant factor smaller than the previous term—then $T(n) = O(f(n))$. In this case, the sum is dominated by the value at the root of the recursion tree.
- **Equal:** If all terms in the series are equal, we immediately have $T(n) = O(f(n) \cdot L) = O(f(n) \log n)$. (The constant c vanishes into the $O(\cdot)$ notation.)
- **Increasing:** If the series *grows exponentially*—every term is a constant factor larger than the previous term—then $T(n) = O(n^{\log_c r})$. In this case, the sum is dominated by the number of leaves in the recursion tree.

In the first and third cases, only the largest term in the geometric series matters; all other terms are swallowed up by the $O(\cdot)$ notation. In the decreasing case, we don't even have to compute L ; the asymptotic upper bound would still hold if the recursion tree were infinite!

As an elementary example, if we draw out the first few levels of the recursion tree for the (simplified) mergesort recurrence $T(n) = 2T(n/2) + O(n)$, we discover that all levels are equal, which immediately implies $T(n) = O(n \log n)$.

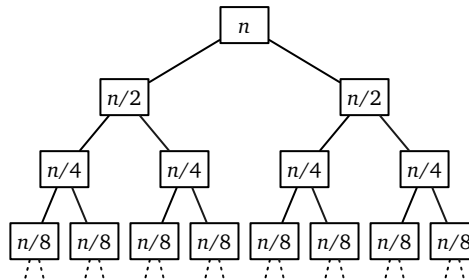


Figure 1.10. The recursion tree for mergesort

The recursion tree technique can also be used for algorithms where the recursive subproblems have different sizes. For example, if we could somehow implement quicksort so that the pivot *always* lands in the middle third of the sorted array, the worst-case running time would satisfy the recurrence

$$T(n) \leq T(n/3) + T(2n/3) + O(n).$$

This recurrence might look scary, but it's actually pretty tame. If we draw out a few levels of the resulting recursion tree, we quickly realize that the sum of values on any level is *at most* n —deeper levels might be missing some nodes—and the entire tree has depth $\log_{3/2} n = O(\log n)$. It immediately follows that $T(n) = O(n \log n)$. (Moreover, the number of *full* levels in the recursion

tree is $\log_3 n = \Omega(\log n)$, so this conservative analysis can be improved by at most a constant factor, which for our purposes means not at all.) The fact that the recursion tree is unbalanced simply doesn't matter.

As a more extreme example, the worst-case recurrence for quicksort $T(n) = T(n-1) + T(1) + O(n)$ gives us a completely unbalanced recursion tree, where one child of each internal node is a leaf. The level-by-level sum doesn't fall into any of our three default categories, but we can still derive the solution $T(n) = O(n^2)$ by observing that every level value is at most n and there are at most n levels. (Again, this conservative analysis is tight, because $n/2$ levels each have value at least $n/2$.)

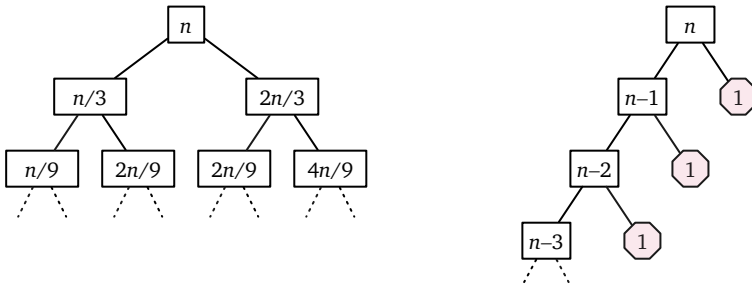


Figure 1.11. Recursion trees for quicksort with good pivots (left) and with worst-case pivots (right)

♥ Ignoring Floors and Ceilings Is Okay, Honest

Careful readers might object that our analysis brushes an important detail under the rug. The running time of mergesort doesn't *really* obey the recurrence $T(n) = 2T(n/2) + O(n)$; after all, the input size n might be odd, and what could it possibly mean to sort an array of size $42\frac{1}{2}$ or $17\frac{7}{8}$? The actual mergesort recurrence is somewhat messier:

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n).$$

Sure, we could check that $T(n) = O(n \log n)$ using induction, but the necessary calculations would be awful. Fortunately, there is a simple technique for removing floors and ceilings from recurrences, called *domain transformation*.

- First, because we are deriving an upper bound, we can safely overestimate $T(n)$, once by pretending that the two subproblem sizes are equal, and again to eliminate the ceiling:⁸

$$T(n) \leq 2T(\lceil n/2 \rceil) + n \leq 2T(n/2 + 1) + n.$$

⁸Formally, we are treating T as a function over the *reals*, not just over the integers, that satisfies the given recurrence with the base case $T(n) = C$ for all $n \leq n_0$, for some real numbers $C \geq 0$ and $n_0 > 0$ whose values don't matter. If n happens to be an integer, then $T(n)$ coincides with the running time of an algorithm on an input of size n , but that doesn't matter, either.

- Second, we define a new function $S(n) = T(n + \alpha)$, choosing the constant α so that $S(n)$ satisfies the simpler recurrence $S(n) \leq 2S(n/2) + O(n)$. To find the correct constant α , we derive a recurrence for S from our given recurrence for T :

$$\begin{aligned}
 S(n) &= T(n + \alpha) && \text{[definition of } S\text{]} \\
 &\leq 2T(n/2 + \alpha/2 + 1) + n + \alpha && \text{[recurrence for } T\text{]} \\
 &= 2S(n/2 - \alpha/2 + 1) + n + \alpha && \text{[definition of } S\text{]}
 \end{aligned}$$

Setting $\alpha = 2$ simplifies this recurrence to $S(n) \leq 2S(n/2) + n + 2$, which is exactly what we wanted.

- Finally, the recursion tree method implies $S(n) = O(n \log n)$, and therefore

$$T(n) = S(n - 2) = O((n - 2) \log(n - 2)) = O(n \log n),$$

exactly as promised.

Similar domain transformations can be used to remove floors, ceilings, and even lower order terms from *any* divide and conquer recurrence. But now that we realize this, we don't need to bother grinding through the details ever again! From now on, faced with any divide-and-conquer recurrence, I will silently brush floors and ceilings and lower-order terms under the rug, and I encourage you to do the same.

♥1.8 Linear-Time Selection

During our discussion of quicksort, I claimed in passing that we can find the median of an unsorted array in linear time. The first such algorithm was discovered by Manuel Blum, Bob Floyd, Vaughan Pratt, Ron Rivest, and Bob Tarjan in the early 1970s. Their algorithm actually solves the more general problem of selecting the k th smallest element in an n -element array, given the array and the integer k as input, using a variant of an algorithm called *quickselect* or *one-armed quicksort*. Quickselect was first described by Tony Hoare in 1961, literally on the same page where he first published quicksort.

Quickselect

The generic quickselect algorithm chooses a pivot element, partitions the array using the same **PARTITION** subroutine as **QUICKSORT**, and then recursively searches *only one of the two subarrays*, specifically, the one that contains the k th smallest element of the original input array. Pseudocode for quickselect is given in Figure 1.12.

```

QUICKSELECT( $A[1..n]$ ,  $k$ ):
  if  $n = 1$ 
    return  $A[1]$ 
  else
    Choose a pivot element  $A[p]$ 
     $r \leftarrow \text{PARTITION}(A[1..n], p)$ 
    if  $k < r$ 
      return QUICKSELECT( $A[1..r-1]$ ,  $k$ )
    else if  $k > r$ 
      return QUICKSELECT( $A[r+1..n]$ ,  $k-r$ )
    else
      return  $A[r]$ 

```

Figure 1.12. Quickselect, or one-armed quicksort

This algorithm has two important features. First, just like quicksort, the correctness of quickselect does not depend on how the pivot is chosen. Second, even if we really only care about selecting medians (the special case $k = n/2$), Hoare's recursive strategy requires us to consider the more general selection problem; the median of the input array $A[1..n]$ is almost never the median of either of the two smaller subarrays $A[1..r-1]$ or $A[r+1..n]$.

The worst-case running time of QUICKSELECT obeys a recurrence similar to QUICKSORT. We don't know the value of r , or which of the two subarrays we'll recursively search, so we have to assume the worst.

$$T(n) \leq \max_{1 \leq r \leq n} \max \{T(r-1), T(n-r)\} + O(n)$$

We can simplify the recurrence slightly by letting ℓ denote the length of the recursive subproblem:

$$T(n) \leq \max_{0 \leq \ell \leq n-1} T(\ell) + O(n)$$

If the chosen pivot element is always either the smallest or largest element in the array, the recurrence simplifies to $T(n) = T(n-1) + O(n)$, which implies $T(n) = O(n^2)$. (The recursion tree for this recurrence is just a simple path.)

Good pivots

We could avoid this quadratic worst-case behavior if we could somehow magically choose a good pivot, meaning $\ell \leq \alpha n$ for some constant $\alpha < 1$. In this case, the recurrence would simplify to

$$T(n) \leq T(\alpha n) + O(n).$$

This recurrence expands into a decreasing geometric series, which is dominated by its largest term, so $T(n) = O(n)$. (Again, the recursion tree is just a simple path. The constant in the $O(n)$ running time depends on the constant α .)

In other words, if we could somehow quickly find an element that's even *close* to the median in linear time, we could find the *exact* median in linear time. So now all we need is an Approximate Median Fairy. The Blum-Floyd-Pratt-Rivest-Tarjan algorithm chooses a good quickselect **pivot by recursively computing the median of a carefully-chosen subset of the input array**. The Approximate Median Fairy is just the Recursion Fairy in disguise!

Specifically, we divide the input array into $\lceil n/5 \rceil$ blocks, each containing exactly 5 elements, except possibly the last. (If the last block isn't full, just throw in a few ∞ s.) We compute the median of each block by brute force, collect those medians into a new array $M[1..\lceil n/5 \rceil]$, and then **recursively compute the median of this new array**. Finally, we use the median of the block medians (called "*mom*" in the pseudocode below) as the quickselect pivot.

```

MOMSELECT( $A[1..n], k$ ):
  if  $n \leq 25$   ⟨⟨or whatever⟩⟩
    use brute force
  else
     $m \leftarrow \lceil n/5 \rceil$ 
    for  $i \leftarrow 1$  to  $m$ 
       $M[i] \leftarrow \text{MEDIANOF FIVE}(A[5i-4..5i])$   ⟨⟨Brute force!⟩⟩
     $mom \leftarrow \text{MOMSELECT}(M[1..m], \lceil m/2 \rceil)$   ⟨⟨Recursion!⟩⟩
     $r \leftarrow \text{PARTITION}(A[1..n], mom)$ 
    if  $k < r$ 
      return MOMSELECT( $A[1..r-1], k$ )  ⟨⟨Recursion!⟩⟩
    else if  $k > r$ 
      return MOMSELECT( $A[r+1..n], k-r$ )  ⟨⟨Recursion!⟩⟩
    else
      return  $mom$ 

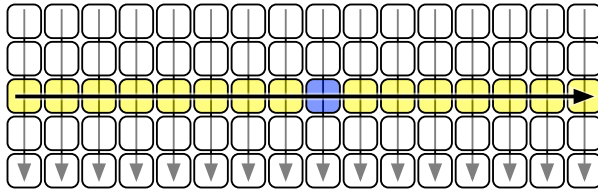
```

MOMSELECT uses recursion for two different purposes; the first time to **choose a pivot element (*mom*)**, and the second time to search through the **entries on one side of that pivot**.

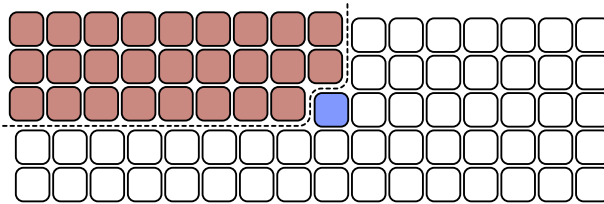
Analysis

But why is this fast? The first key insight is that **the median of medians is a good pivot**. Mom is larger than $\lfloor \lceil n/5 \rceil / 2 \rfloor - 1 \approx n/10$ block medians, and each **block median is larger than two other elements in its block**. Thus, mom is bigger than at least **$3n/10$ elements in the input array**; symmetrically, mom is smaller than at least $3n/10$ elements. Thus, in the worst case, the second recursive call searches an array of size at most **$7n/10$** .

We can visualize the algorithm's behavior by drawing the input array as a $5 \times \lceil n/5 \rceil$ grid, which each column represents five consecutive elements. For purposes of illustration, imagine that we sort every column from top down, and then we sort the columns by their middle element. (Let me emphasize that *the algorithm does not actually do this!*) In this arrangement, the median-of-medians is the element closest to the center of the grid.



The left half of the first three rows of the grid contains $3n/10$ elements, each of which is smaller than mom. If the element we're looking for is larger than mom, our algorithm will throw away *everything* smaller than mom, including those $3n/10$ elements, before recursing. Thus, the input to the recursive subproblem contains at most $7n/10$ elements. A symmetric argument implies that if our target element is smaller than mom, we discard at least $3n/10$ elements larger than mom, so the input to our recursive subproblem has at most $7n/10$ elements.



Okay, so mom is a good pivot, but our algorithm still makes *two* recursive calls instead of just one; how do we prove linear time? The second key insight is that the *total size of the two recursive subproblems is a constant factor smaller than the size of the original input array*. The worst-case running time of the algorithm obeys the recurrence

$$T(n) \leq T(n/5) + T(7n/10) + O(n).$$

If we draw out the recursion tree for this recurrence, we observe that the total work at each level of the recursion tree is at most $9/10$ the total work at the previous level. Thus, the level sums decay exponentially, giving us the solution $T(n) = O(n)$. (Again, the fact that the recursion tree is unbalanced is completely immaterial.) Hooray! Thanks, Mom!

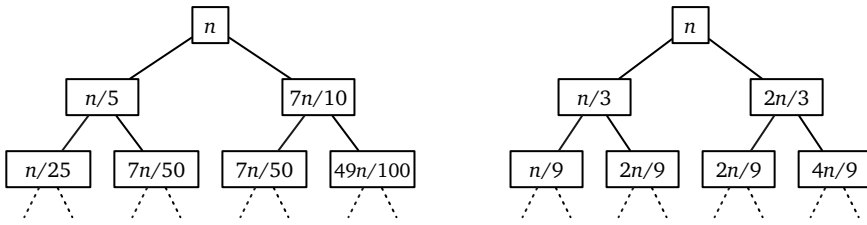


Figure 1.13. The recursion trees for **MOMSELECT** and a similar selection algorithm with blocks of size 3

Sanity Checking

At this point, many students ask about that magic constant 5. Why did we choose that particular block size? The answer is that 5 **is the smallest odd block size that gives us exponential decay in the recursion-tree analysis!** (Even block sizes introduce additional complications.) If we had used blocks of size 3 instead, the running-time recurrence would be

$$T(n) \leq T(n/3) + T(2n/3) + O(n).$$

We've seen this recurrence before! Every level of the recursion tree has total value *at most* n , and the **depth of the recursion tree is $\log_{3/2} n = O(\log n)$** , so the solution to this recurrence is **$T(n) \leq O(n \log n)$** . (Moreover, this analysis is tight, because the recursion tree has $\log_3 n$ complete levels.) Median-of-medians selection using **3-element blocks is no faster than sorting**.

Finer analysis reveals that the **constant hidden by the $O()$ notation is quite large**, even if we count only comparisons. Selecting the median of 5 elements requires at most 6 comparisons, **so we need at most $6n/5$ comparisons to set up the recursive subproblem**. Naïvely partitioning the array after the recursive call would require $n - 1$ comparisons, but we already know $3n/10$ elements larger than the pivot and $3n/10$ elements smaller than the pivot, so partitioning actually requires only **$2n/5$ additional comparisons**. Thus, a more precise recurrence for the worst-case number of comparisons is

$$T(n) \leq T(n/5) + T(7n/10) + 8n/5.$$

The recursion tree method implies the upper bound

$$T(n) \leq \frac{8n}{5} \sum_{i \geq 0} \left(\frac{9}{10} \right)^i = \frac{8n}{5} \cdot 10 = 16n.$$

In practice, median-of-medians selection is not as slow as this worst-case analysis predicts—getting a worst-case pivot at every level of recursion is incredibly unlikely—but it is still slower than sorting for even moderately large arrays.⁹

⁹In fact, the right way to choose the pivot element in practice is to choose it *uniformly at random*. Then the expected number of comparisons required to find the median is at most $4n$. See my randomized algorithms lecture notes at <http://algorithms.wtf> for more details.

1.9 Fast Multiplication

In the previous chapter, we saw two ancient algorithms for multiplying two n -digit numbers in $O(n^2)$ time: the grade-school lattice algorithm and the Egyptian peasant algorithm.

Maybe we can get a more efficient algorithm by splitting the digit arrays in half and exploiting the following identity:

$$(10^m a + b)(10^m c + d) = 10^{2m} ac + 10^m (bc + ad) + bd$$

This recurrence immediately suggests the following divide-and-conquer algorithm to multiply two n -digit numbers x and y . Each of the four sub-products ac , bc , ad , and bd is computed recursively, but the multiplications in the last line are *not* recursive, because we can multiply by a power of ten by shifting the digits to the left and filling in the correct number of zeros, all in $O(n)$ time.

```

SPLITMULTIPLY( $x, y, n$ ):
  if  $n = 1$ 
    return  $x \cdot y$ 
  else
     $m \leftarrow \lceil n/2 \rceil$ 
     $a \leftarrow \lfloor x/10^m \rfloor$ ;  $b \leftarrow x \bmod 10^m$        $\langle\langle x = 10^m a + b \rangle\rangle$ 
     $c \leftarrow \lfloor y/10^m \rfloor$ ;  $d \leftarrow y \bmod 10^m$      $\langle\langle y = 10^m c + d \rangle\rangle$ 
     $e \leftarrow \text{SPLITMULTIPLY}(a, c, m)$ 
     $f \leftarrow \text{SPLITMULTIPLY}(b, d, m)$ 
     $g \leftarrow \text{SPLITMULTIPLY}(b, c, m)$ 
     $h \leftarrow \text{SPLITMULTIPLY}(a, d, m)$ 
    return  $10^{2m} e + 10^m (g + h) + f$ 

```

Correctness of this algorithm follows easily by induction. The running time for this algorithm follows the recurrence

$$T(n) = 4T(\lceil n/2 \rceil) + O(n).$$

The recursion tree method transforms this recurrence into an *increasing* geometric series, which implies $T(n) = O(n^{\log_2 4}) = O(n^2)$. In fact, this algorithm multiplies each digit of x with each digit of y , just like the lattice algorithm. So I guess that didn't work. Too bad. It was a nice idea.

In the mid-1950s, Andrei Kolmogorov, one of the giants of 20th century mathematics, publicly conjectured that there is *no* algorithm to multiply two n -digit numbers in subquadratic time. Kolmogorov organized a seminar at Moscow University in 1960, where he restated his “ n^2 conjecture” and posed several related problems that he planned to discuss at future meetings. Almost exactly a week later, a 23-year-old student named Anatoliĭ Karatsuba presented Kolmogorov with a remarkable counterexample. According to Karatsuba himself,

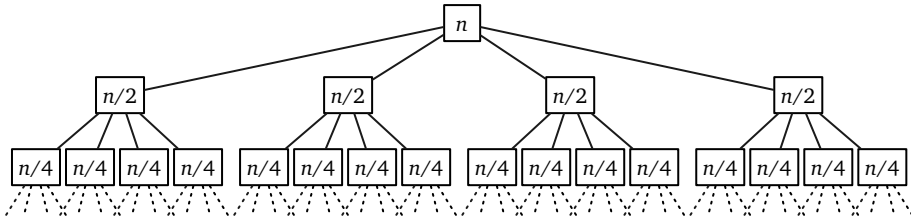


Figure 1.14. The recursion tree for naïve divide-and-conquer multiplication

After the seminar I told Kolmogorov about the new algorithm and about the disproof of the n^2 conjecture. Kolmogorov was very agitated because this contradicted his very plausible conjecture. At the next meeting of the seminar, Kolmogorov himself told the participants about my method, and at that point the seminar was terminated.

Karatsuba observed that the middle coefficient $bc + ad$ can be computed from the other two coefficients ac and bd using only *one* more recursive multiplication, via the following algebraic identity:

$$ac + bd - (a - b)(c - d) = bc + ad$$

This trick lets us replace the four recursive calls in the previous algorithm with only three recursive calls, as shown below:

```

FASTMULTIPLY( $x, y, n$ ):
  if  $n = 1$ 
    return  $x \cdot y$ 
  else
     $m \leftarrow \lceil n/2 \rceil$ 
     $a \leftarrow \lfloor x/10^m \rfloor$ ;  $b \leftarrow x \bmod 10^m$      $\langle\langle x = 10^m a + b \rangle\rangle$ 
     $c \leftarrow \lfloor y/10^m \rfloor$ ;  $d \leftarrow y \bmod 10^m$      $\langle\langle y = 10^m c + d \rangle\rangle$ 
     $e \leftarrow \text{FASTMULTIPLY}(a, c, m)$ 
     $f \leftarrow \text{FASTMULTIPLY}(b, d, m)$ 
     $g \leftarrow \text{FASTMULTIPLY}(a - b, c - d, m)$ 
    return  $10^{2m}e + 10^m(e + f - g) + f$ 

```

The running time of Karatsuba's FASTMULTIPLY algorithm follows the recurrence

$$T(n) \leq 3T(\lceil n/2 \rceil) + O(n)$$

Once again, the recursion tree method transforms this recurrence into an increasing geometric series, but the new solution is only $T(n) = O(n^{\log_2 3}) = O(n^{1.58496})$, a significant improvement over our earlier quadratic time bound.¹⁰

¹⁰My presentation simplifies the actual history slightly. In fact, Karatsuba proposed an algorithm based on the formula $(a + b)(c + d) - ac - bd = bc + ad$. This algorithm also runs in $O(n^{\lg 3})$ time, but the actual recurrence is slightly messier: $a - b$ and $c - d$ are still m -digit numbers, but $a + b$ and $c + d$ might each have $m + 1$ digits. The simplification presented here is due to Donald Knuth.

Karatsuba’s algorithm arguably launched the design and analysis of algorithms as a formal field of study.

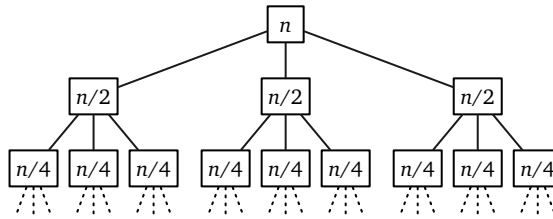


Figure 1.15. The recursion tree for Karatsuba’s divide-and-conquer multiplication algorithm

We can take Karatsuba’s idea even further, splitting the numbers into more pieces and combining them in more complicated ways, to obtain even faster multiplication algorithms. Andrei Toom discovered an infinite family of algorithms that split any integer into k parts, each with n/k digits, and then compute the product using only $2k - 1$ recursive multiplications; Toom’s algorithms were further simplified by Stephen Cook in his PhD thesis. For any fixed k , the Toom-Cook algorithm runs in $O(n^{1+1/(\lg k)})$ time, where the hidden constant in the $O(\cdot)$ notation depends on k .

Ultimately, this divide-and-conquer strategy led Gauss (yes, really) to the discovery of the **Fast Fourier transform**.¹¹ The basic FFT algorithm itself runs in $O(n \log n)$ time; however, using FFTs for integer multiplication incurs some small additional overhead. The first FFT-based integer multiplication algorithm, published by Arnold Schönhage and Volker Strassen in 1971, runs in $O(n \log n \log \log n)$ time. Schönhage-Strassen remained the theoretically fastest integer multiplication algorithm for several decades, before Martin Fürer discovered the first of a long series of technical improvements. Finally, in 2019, David Harvey and Joris van der Hoeven published an algorithm that runs in $O(n \log n)$ time.¹²

1.10 Exponentiation

Given a number a and a positive integer n , suppose we want to compute a^n . The standard naïve method is a simple for-loop that performs $n - 1$ multiplications by a :

¹¹See <http://algorithms.wtf> for lecture notes on Fast Fourier transforms.

¹²Schönhage-Strassen is actually the fastest algorithm *in practice* for multiplying integers with more than about 75000 digits; the more recent algorithms of Fürer, Harvey, van der Hoeven, and others would be faster “in practice” only for integers with more digits than there are particles in the universe.

<u>SLOWPOWER(a, n):</u> $x \leftarrow a$ for $i \leftarrow 2$ to n $x \leftarrow x \cdot a$ return x

This iterative algorithm requires n multiplications.

The input parameter a could be an integer, or a rational, or a floating point number. In fact, it doesn't need to be a number at all, as long as it's something that we know how to multiply. For example, the same algorithm can be used to compute powers modulo some finite number (an operation commonly used in cryptography algorithms) or to compute powers of matrices (an operation used to evaluate recurrences and to compute shortest paths in graphs). Because we don't know what kind of object we're multiplying, we *can't* know how much time a single multiplication requires, so we're forced to analyze the running time in terms of the number of multiplications.

There is a much faster divide-and-conquer method, originally proposed by the Indian prosodist Piṅgala in the 2nd century BCE, which uses the following simple recursive formula:

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{otherwise} \end{cases}$$

<u>PIṆGALAPOWER(a, n):</u> if $n = 1$ return a else $x \leftarrow \text{PIṆGALAPOWER}(a, \lfloor n/2 \rfloor)$ if n is even return $x \cdot x$ else return $x \cdot x \cdot a$
--

The total number of multiplications performed by this algorithm satisfies the recurrence $T(n) \leq T(n/2) + 2$. The recursion-tree method immediately give us the solution $T(n) = O(\log n)$.

A nearly identical exponentiation algorithm can also be derived directly from the Egyptian peasant multiplication algorithm from the previous chapter, by replacing addition with multiplication (and in particular, replacing duplation with squaring).

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^2)^{n/2} & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{otherwise} \end{cases}$$

```
PEASANTPOWER( $a, n$ ):  
  if  $n = 1$   
    return  $a$   
  else if  $n$  is even  
    return PEASANTPOWER( $a^2, n/2$ )  
  else  
    return PEASANTPOWER( $a^2, \lfloor n/2 \rfloor$ ) ·  $a$ 
```

This algorithm—which might reasonably be called “squaring and mediation”—also performs only $O(\log n)$ multiplications.

Both of these algorithms are asymptotically optimal; any algorithm that computes a^n must perform at least $\Omega(\log n)$ multiplications, because each multiplication at most doubles the largest power computed so far. In fact, when n is a power of two, both of these algorithms require exactly $\log_2 n$ multiplications, which is *exactly* optimal. However, there are slightly faster methods for other values of n . For example, PINGALAPOWER and PEASANTPOWER each compute a^{15} using six multiplications, but in fact only five multiplications are necessary:

- Pingala: $a \rightarrow a^2 \rightarrow a^3 \rightarrow a^6 \rightarrow a^7 \rightarrow a^{14} \rightarrow a^{15}$
- Peasant: $a \rightarrow a^2 \rightarrow a^4 \rightarrow a^8 \rightarrow a^{12} \rightarrow a^{14} \rightarrow a^{15}$
- Optimal: $a \rightarrow a^2 \rightarrow a^3 \rightarrow a^5 \rightarrow a^{10} \rightarrow a^{15}$

It is a long-standing open question whether the absolute minimum number of multiplications for a given exponent n can be computed efficiently.

Exercises

Tower of Hanoi

1. Prove that the original recursive Tower of Hanoi algorithm performs *exactly* the same sequence of moves—the same disks, to and from the same pegs, in the same order—as each of the following non-recursive algorithms. The pegs are labeled 0, 1, and 2, and our problem is to move a stack of n disks from peg 0 to peg 2 (as shown on page 24).
 - (a) If n is even, swap pegs 1 and 2. At the i th step, make the only legal move that avoids peg $i \bmod 3$. If there is no legal move, then all disks are on peg $i \bmod 3$, and the puzzle is solved.
 - (b) For the first move, move disk 1 to peg 1 if n is even and to peg 2 if n is odd. Then repeatedly make the only legal move that involves a different disk from the previous move. If no such move exists, the puzzle is solved.
 - (c) Pretend that disks $n + 1$, $n + 2$, and $n + 3$ are at the bottom of pegs 0, 1, and 2, respectively. Repeatedly make the only legal move that satisfies the following constraints, until no such move is possible.

- Do not place an odd disk directly on top of another odd disk.
 - Do not place an even disk directly on top of another even disk.
 - Do not undo the previous move.
- (d) Let $\rho(n)$ denote the smallest integer k such that $n/2^k$ is not an integer. For example, $\rho(42) = 2$, because $42/2^1$ is an integer but $42/2^2$ is not. (Equivalently, $\rho(n)$ is one more than the position of the least significant 1 in the binary representation of n .) Because its behavior resembles the marks on a ruler, $\rho(n)$ is sometimes called the *ruler function*.

```

RULERHANOI( $n$ ):
   $i \leftarrow 1$ 
  while  $\rho(i) \leq n$ 
    if  $n - i$  is even
      move disk  $\rho(i)$  forward     $\langle\langle 0 \rightarrow 1 \rightarrow 2 \rightarrow 0 \rangle\rangle$ 
    else
      move disk  $\rho(i)$  backward   $\langle\langle 0 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rangle\rangle$ 
   $i \leftarrow i + 1$ 

```

2. The Tower of Hanoi is a relatively recent descendant of a much older mechanical puzzle known as the Chinese linked rings, Baguenaudier, Cardan's Rings, Meleda, Patience, Tiring Irons, Prisoner's Lock, Spin-Out, and many other names. This puzzle was already well known in both China and Europe by the 16th century. The Italian mathematician Luca Pacioli described the 7-ring puzzle and its solution in his unpublished treatise *De Viribus Quantitatis*, written between 1498 and 1506;¹³ only a few years later, the Ming-dynasty poet Yang Shen described the 9-ring puzzle as “a toy for women and children”. The puzzle is apocryphally attributed to a 2nd-century Chinese general, who gave the puzzle to his wife to occupy her time while he was away at war.

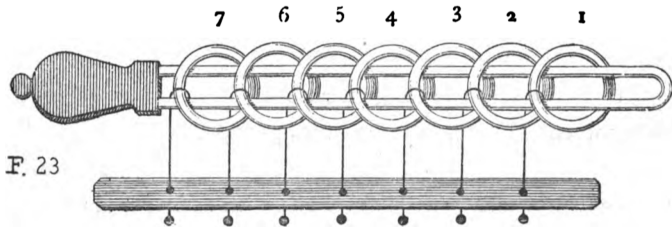


Figure 1.16. The 7-ring Baguenaudier, from *Récréations Mathématiques* by Édouard Lucas (1891) (See Image Credits at the end of the book.)

¹³*De Viribus Quantitatis* [On the Powers of Numbers] is an important early work on recreational mathematics and perhaps the oldest surviving treatise on magic. Pacioli is better known for *Summa de Arithmetica*, a near-complete encyclopedia of late 15th-century mathematics, which included the first description of double-entry bookkeeping.

The Baguenaudier puzzle has many physical forms, but one of the most common consists of a long metal loop and several rings, which are connected to a solid base by movable rods. The loop is initially threaded through the rings as shown in Figure 1.16; the goal of the puzzle is to remove the loop.

More abstractly, we can model the puzzle as a sequence of bits, one for each ring, where the i th bit is 1 if the loop passes through the i th ring and 0 otherwise. (Here we index the rings from right to left, as shown in Figure 1.16.) The puzzle allows two legal moves:

- You can always flip the 1st (= rightmost) bit.
- If the bit string ends with exactly z 0s, you can flip the $(z + 2)$ th bit.

The goal of the puzzle is to transform a string of n 1s into a string of n 0s. For example, the following sequence of 21 moves solves the 5-ring puzzle:

$$\begin{aligned} 11111 &\xrightarrow{1} 11110 \xrightarrow{3} 11010 \xrightarrow{1} 11011 \xrightarrow{2} 11001 \xrightarrow{1} 11000 \\ &\xrightarrow{5} 01000 \xrightarrow{1} 01001 \xrightarrow{2} 01011 \xrightarrow{1} 01010 \xrightarrow{3} 01110 \\ &\xrightarrow{1} 01111 \xrightarrow{2} 01101 \xrightarrow{1} 01100 \xrightarrow{4} 00100 \xrightarrow{1} 00101 \\ &\xrightarrow{2} 00111 \xrightarrow{1} 00110 \xrightarrow{3} 00010 \xrightarrow{1} 00011 \xrightarrow{2} 00001 \xrightarrow{1} 00000 \end{aligned}$$

- ♦(a) Call a sequence of moves *reduced* if no move is the inverse of the previous move. Prove that for any non-negative integer n , there is *exactly one* reduced sequence of moves that solves the n -ring Baguenaudier puzzle. [Hint: This problem is much easier if you're already familiar with graphs.]
 - (b) Describe an algorithm to solve the Baguenaudier puzzle. Your input is the number of rings n ; your algorithm should print a reduced sequence of moves that solves the puzzle. For example, given the integer 5 as input, your algorithm should print the sequence 1, 3, 1, 2, 1, 5, 1, 2, 1, 3, 1, 2, 1, 4, 1, 2, 1, 3, 1, 2, 1.
 - (c) *Exactly* how many moves does your algorithm perform, as a function of n ? Prove your answer is correct.
3. A less familiar chapter in the Tower of Hanoi's history is its brief relocation of the temple from Benares to Pisa in the early 13th century.¹⁴ The relocation was organized by the wealthy merchant-mathematician Leonardo Fibonacci, at the request of the Holy Roman Emperor Frederick II, who had heard reports of the temple from soldiers returning from the Crusades. The Towers of Pisa and their attendant monks became famous, helping to establish Pisa as a dominant trading center on the Italian peninsula.

¹⁴Portions of this story are actually true.

Unfortunately, almost as soon as the temple was moved, one of the diamond needles began to lean to one side. To avoid the possibility of the leaning tower falling over from too much use, Fibonacci convinced the priests to adopt a more relaxed rule: **Any number of disks on the leaning needle can be moved together to another needle in a single move.** It was still forbidden to place a larger disk on top of a smaller disk, and disks had to be moved one at a time *onto* the leaning needle or between the two vertical needles.

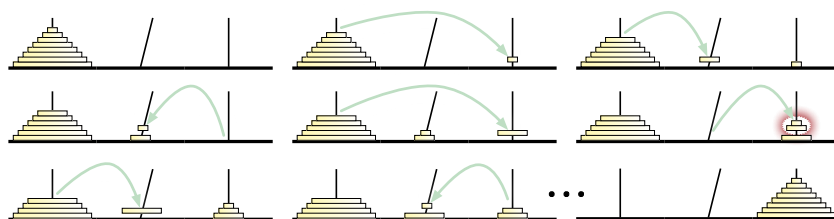


Figure 1.17. The Towers of Pisa. In the fifth move, two disks are taken off the leaning needle.

Thanks to Fibonacci's new rule, the priests could bring about the end of the universe somewhat faster from Pisa than they could from Benares. Fortunately, the temple was moved from Pisa back to Benares after the newly crowned Pope Gregory IX excommunicated Frederick II, making the local priests less sympathetic to hosting foreign heretics with strange mathematical habits. Soon afterward, a bell tower was erected on the spot where the temple once stood; it too began to lean almost immediately.

Describe an algorithm to transfer a stack of n disks from one *vertical* needle to the other *vertical* needle, using the smallest possible number of moves. *Exactly* how many moves does your algorithm perform?

4. Consider the following restricted variants of the Tower of Hanoi puzzle. In each problem, the pegs are numbered 0, 1, and 2, and your task is to move a stack of n disks from peg 0 to peg 2, exactly as in problem 1.
 - (a) Suppose you are forbidden to move any disk directly between peg 1 and peg 2; *every* move must involve peg 0. Describe an algorithm to solve this version of the puzzle in as few moves as possible. *Exactly* how many moves does your algorithm make?
 - ♥♣ (b) Suppose you are only allowed to move disks from peg 0 to peg 2, from peg 2 to peg 1, or from peg 1 to peg 0. Equivalently, suppose the pegs are arranged in a circle and numbered in clockwise order, and you are only allowed to move disks counterclockwise. Describe an algorithm to solve this version of the puzzle in as few moves as possible. How many moves does your algorithm make?

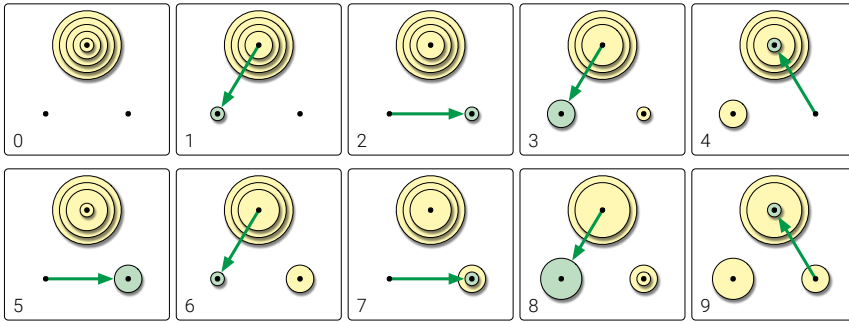


Figure 1.18. The first several moves in a counterclockwise Towers of Hanoi solution.

- ♥♣(c) Finally, suppose your only restriction is that you may never move a disk directly from peg 0 to peg 2. Describe an algorithm to solve this version of the puzzle in as few moves as possible. How many moves does your algorithm make? *[Hint: Matrices! This variant is considerably harder to analyze than the other two.]*
5. Consider the following more complex variant of the Tower of Hanoi puzzle. The puzzle has a row of k pegs, numbered from 1 to k . In a single turn, you are allowed to move the smallest disk on peg i to either peg $i - 1$ or peg $i + 1$, for any index i ; as usual, you are not allowed to place a bigger disk on a smaller disk. Your mission is to move a stack of n disks from peg 1 to peg k .
- (a) Describe a recursive algorithm for the case $k = 3$. *Exactly* how many moves does your algorithm make? (This is exactly the same as problem 4(a).)
- (b) Describe a recursive algorithm for the case $k = n + 1$ that requires at most $O(n^3)$ moves. *[Hint: Use part (a).]*
- ♥(c) Describe a recursive algorithm for the case $k = n + 1$ that requires at most $O(n^2)$ moves. *[Hint: Don't use part (a).]*
- ♥(d) Describe a recursive algorithm for the case $k = \sqrt{n}$ that requires at most a polynomial number of moves. (Which polynomial??)
- ♥(e) Describe and analyze a recursive algorithm for arbitrary n and k . How small must k be (as a function of n) so that the number of moves is bounded by a polynomial in n ?

Recursion Trees

6. Use recursion trees to solve each of the following recurrences.

$$A(n) = 2A(n/4) + \sqrt{n} \quad B(n) = 2B(n/4) + n \quad C(n) = 2C(n/4) + n^2$$

$$D(n) = 3D(n/3) + \sqrt{n} \quad E(n) = 3E(n/3) + n \quad F(n) = 3F(n/3) + n^2$$

$$G(n) = 4G(n/2) + \sqrt{n} \quad H(n) = 4H(n/2) + n \quad I(n) = 4I(n/2) + n^2$$

7. Use recursion trees to solve each of the following recurrences.

$$(j) \quad J(n) = J(n/2) + J(n/3) + J(n/6) + n$$

$$(k) \quad K(n) = K(n/2) + 2K(n/3) + 3K(n/4) + n^2$$

$$(l) \quad L(n) = L(n/15) + L(n/10) + 2L(n/6) + \sqrt{n}$$

♥8. Use recursion trees to solve each of the following recurrences.

$$(m) \quad M(n) = 2M(n/2) + O(n \log n)$$

$$(n) \quad N(n) = 2N(n/2) + O(n/\log n)$$

$$(p) \quad P(n) = \sqrt{n}P(\sqrt{n}) + n$$

$$(q) \quad Q(n) = \sqrt{2n}Q(\sqrt{2n}) + \sqrt{n}$$

Sorting

9. Suppose you are given a stack of n pancakes of different sizes. You want to sort the pancakes so that smaller pancakes are on top of larger pancakes. The only operation you can perform is a *flip*—insert a spatula under the top k pancakes, for some integer k between 1 and n , and flip them all over.

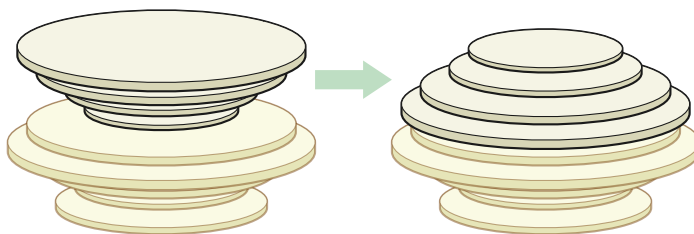


Figure 1.19. Flipping the top four pancakes.

(a) Describe an algorithm to sort an arbitrary stack of n pancakes using $O(n)$ flips. *Exactly* how many flips does your algorithm perform in the worst case?¹⁵ [Hint: This problem has nothing to do with the Tower of Hanoi.]

¹⁵The exact worst-case optimal number of flips required to sort n pancakes (either burned or unburned) is an long-standing open problem; just do the best you can.

- (b) For every positive integer n , describe a stack of n pancakes that requires $\Omega(n)$ flips to sort.
 - (c) Now suppose one side of each pancake is burned. Describe an algorithm to sort an arbitrary stack of n pancakes, so that the burned side of every pancake is facing down, using $O(n)$ flips. *Exactly* how many flips does your algorithm perform in the worst case?
10. Recall that the *median-of-three* heuristic examines the first, last, and middle element of the array, and uses the median of those three elements as a quicksort pivot. Prove that quicksort with the median-of-three heuristic requires $\Omega(n^2)$ time to sort an array of size n in the worst case. Specifically, for any integer n , describe a permutation of the integers 1 through n , such that in every recursive call to median-of-three-quicksort, the pivot is always the second smallest element of the array. Designing this permutation requires intimate knowledge of the PARTITION subroutine.
- (a) As a warm-up exercise, assume that the PARTITION subroutine is *stable*, meaning it preserves the existing order of all elements smaller than the pivot, and it preserves the existing order of all elements smaller than the pivot.
 - ♥(b) Assume that the PARTITION subroutine uses the specific algorithm listed on page 29, which is *not* stable.
11. (a) Hey, Moe! Hey, Larry! Prove that the following algorithm actually sorts its input!

```
STOOGESORT( $A[0..n-1]$ ):  
  if  $n = 2$  and  $A[0] > A[1]$   
    swap  $A[0] \leftrightarrow A[1]$   
  else if  $n > 2$   
     $m = \lceil 2n/3 \rceil$   
    STOOGESORT( $A[0..m-1]$ )  
    STOOGESORT( $A[n-m..n-1]$ )  
    STOOGESORT( $A[0..m-1]$ )
```

- (b) Would STOOGESORT still sort correctly if we replaced $m = \lceil 2n/3 \rceil$ with $m = \lfloor 2n/3 \rfloor$? Justify your answer.
 - (c) State a recurrence (including the base case(s)) for the number of comparisons executed by STOOGESORT.
 - (d) Solve the recurrence, and prove that your solution is correct. [Hint: Ignore the ceiling.]
 - (e) Prove that the number of swaps executed by STOOGESORT is at most $\binom{n}{2}$.
12. The following cruel and unusual sorting algorithm was proposed by Gary Miller:

```

CRUEL(A[1 .. n]):
  if n > 1
    CRUEL(A[1 .. n/2])
    CRUEL(A[n/2 + 1 .. n])
    UNUSUAL(A[1 .. n])

```

```

UNUSUAL(A[1..n]):
  if n = 2
    if A[1] > A[2]                                «the only comparison!»
      swap A[1] ↔ A[2]
  else
    for i ← 1 to n/4                               «swap 2nd and 3rd quarters»
      swap A[i + n/4] ↔ A[i + n/2]
    UNUSUAL(A[1..n/2])                             «recurse on left half»
    UNUSUAL(A[n/2 + 1..n])                         «recurse on right half»
    UNUSUAL(A[n/4 + 1..3n/4])                     «recurse on middle half»

```

The comparisons performed by this algorithm do not depend at all on the values in the input array; such a sorting algorithm is called *oblivious*. Assume for this problem that the input size n is always a power of 2.

- (a) Prove by induction that CRUEL correctly sorts any input array. [Hint: Consider an array that contains $n/4$ 1s, $n/4$ 2s, $n/4$ 3s, and $n/4$ 4s. Why is this special case enough?]
 - (b) Prove that CRUEL would *not* correctly sort if we removed the for-loop from UNUSUAL.
 - (c) Prove that CRUEL would *not* correctly sort if we swapped the last two lines of UNUSUAL.
 - (d) What is the running time of UNUSUAL? Justify your answer.
 - (e) What is the running time of CRUEL? Justify your answer.
13. An ***inversion*** in an array $A[1..n]$ is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. The number of inversions in an n -element array is between 0 (if the array is sorted) and $\binom{n}{2}$ (if the array is sorted backward). Describe and analyze an algorithm to count the number of inversions in an n -element array in $O(n \log n)$ time. [Hint: Modify mergesort.]
14. (a) Suppose you are given two sets of n points, one set $\{p_1, p_2, \dots, p_n\}$ on the line $y = 0$ and the other set $\{q_1, q_2, \dots, q_n\}$ on the line $y = 1$. Create a set of n line segments by connect each point p_i to the corresponding point q_i . Describe and analyze a divide-and-conquer algorithm to determine how many pairs of these line segments intersect, in $O(n \log n)$ time. [Hint: See the previous problem.]
- (b) Now suppose you are given two sets $\{p_1, p_2, \dots, p_n\}$ and $\{q_1, q_2, \dots, q_n\}$ of n points *on the unit circle*. Connect each point p_i to the corresponding

point q_i . Describe and analyze a divide-and-conquer algorithm to determine how many pairs of these line segments intersect in $O(n \log^2 n)$ time. [Hint: Use your solution to part (a).]

- ♥(c) Describe an algorithm for part (b) that runs in $O(n \log n)$ time. [Hint: Use your solution from part (b)!]

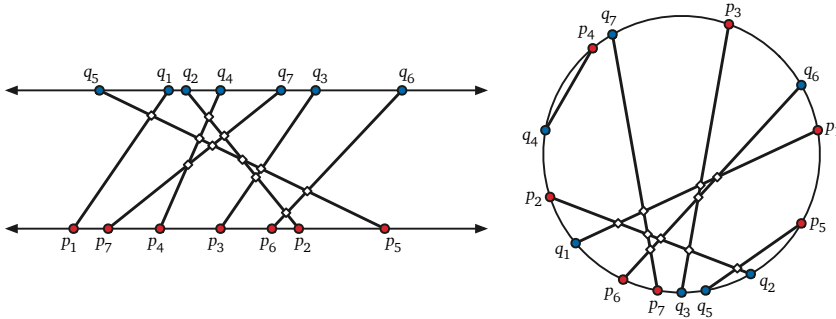


Figure 1.20. Eleven intersecting pairs of segments with endpoints on parallel lines, and ten intersecting pairs of segments with endpoints on a circle.

15. (a) Describe an algorithm that sorts an input array $A[1..n]$ by calling a subroutine $\text{SQRTSORT}(k)$, which sorts the subarray $A[k+1..k+\sqrt{n}]$ in place, given an arbitrary integer k between 0 and $n - \sqrt{n}$ as input. (To simplify the problem, assume that $\sqrt{n}$ is an integer.) Your algorithm is **only** allowed to inspect or modify the input array by calling SQRTSORT ; in particular, your algorithm must not directly compare, move, or copy array elements. How many times does your algorithm call SQRTSORT in the worst case?
- ♣(b) Prove that your algorithm from part (a) is optimal up to constant factors. In other words, if $f(n)$ is the number of times your algorithm calls SQRTSORT , prove that no algorithm can sort using $o(f(n))$ calls to SQRTSORT .
- (c) Now suppose SQRTSORT is implemented recursively, by calling your sorting algorithm from part (a). For example, at the second level of recursion, the algorithm is sorting arrays roughly of size $n^{1/4}$. What is the worst-case running time of the resulting sorting algorithm? (To simplify the analysis, assume that the array size n has the form 2^{2^k} , so that repeated square roots are always integers.)

Selection

16. Suppose we are given a set S of n items, each with a *value* and a *weight*. For any element $x \in S$, we define two subsets

- $S_{<x}$ is the set of elements of S whose value is less than the value of x .
- $S_{>x}$ is the set of elements of S whose value is more than the value of x .

For any subset $R \subseteq S$, let $w(R)$ denote the sum of the weights of elements in R . The **weighted median** of R is any element x such that $w(S_{<x}) \leq w(S)/2$ and $w(S_{>x}) \leq w(S)/2$.

Describe and analyze an algorithm to compute the weighted median of a given weighted set in $O(n)$ time. Your input consists of two unsorted arrays $S[1..n]$ and $W[1..n]$, where for each index i , the i th element has value $S[i]$ and weight $W[i]$. You may assume that all values are distinct and all weights are positive.

17. (a) Describe an algorithm to determine in $O(n)$ time whether an arbitrary array $A[1..n]$ contains more than $n/4$ copies of any value.
- (b) Describe and analyze an algorithm to determine, given an arbitrary array $A[1..n]$ and an integer k , whether A contains more than k copies of any value. Express the running time of your algorithm as a function of both n and k .

Do not use hashing, or radix sort, or any other method that depends on the precise input values, as opposed to their order.

18. Describe an algorithm to compute the median of an array $A[1..5]$ of distinct numbers using at most 6 comparisons. Instead of writing pseudocode, describe your algorithm using a **decision tree**: A binary tree where each internal node contains a comparison of the form “ $A[i] \geq A[j]$?” and each leaf contains an index into the array.

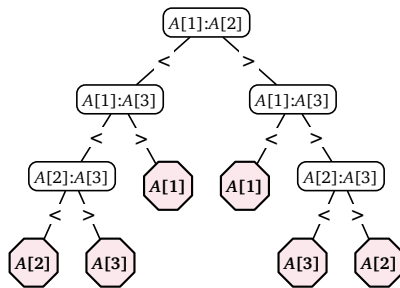


Figure 1.21. Finding the median of a 3-element array using at most 3 comparisons

19. Consider the generalization of the Blum-Floyd-Pratt-Rivest-Tarjan MOM-SELECT algorithm shown in Figure 1.22, which partitions the input array into $\lceil n/b \rceil$ blocks of size b , instead of $\lceil n/5 \rceil$ blocks of size 5, but is otherwise identical.

```

MOMbSELECT( $A[1..n]$ ,  $k$ ):
  if  $n \leq b^2$ 
    use brute force
  else
     $m \leftarrow \lceil n/b \rceil$ 
    for  $i \leftarrow 1$  to  $m$ 
       $M[i] \leftarrow \text{MEDIANOFB}(A[b(i-1)+1..bi])$ 
     $mom_b \leftarrow \text{MOM}_b\text{SELECT}(M[1..m], \lfloor m/2 \rfloor)$ 
     $r \leftarrow \text{PARTITION}(A[1..n], mom_b)$ 
    if  $k < r$ 
      return MOMbSELECT( $A[1..r-1]$ ,  $k$ )
    else if  $k > r$ 
      return MOMbSELECT( $A[r+1..n]$ ,  $k-r$ )
    else
      return  $mom_b$ 

```

Figure 1.22. A parametrized family of selection algorithms; see problem 19.

- (a) State a recurrence for the running time of MOM_bSELECT, assuming that b is a constant (so the subroutine MEDIANOFB runs in $O(1)$ time). In particular, how do the sizes of the recursive subproblems depend on the constant b ? Consider even b and odd b separately.
- (b) What is the worst-case running time of MOM₁SELECT? *[Hint: This is a trick question.]*
- ♣♥(c) What is the worst-case running time of MOM₂SELECT? *[Hint: This is an unfair question!]*
- ♥(d) What is the worst-case running time of MOM₃SELECT? Finding an upper bound on the running time is straightforward; the hard part is showing that this analysis is actually tight. *[Hint: See problem 10.]*
- ♥(e) What is the worst-case running time of MOM₄SELECT? Again, the hard part is showing that the analysis cannot be improved.¹⁶
- (f) For any constants $b \geq 5$, the algorithm MOM_bSELECT runs in $O(n)$ time, but different values of b lead to different constant factors. Let $M(b)$ denote the minimum number of comparisons required to find the median of b numbers. The exact value of $M(b)$ is known only for $b \leq 13$:

b	1	2	3	4	5	6	7	8	9	10	11	12	13
$M(b)$	0	1	3	4	6	8	10	12	14	16	18	20	23

¹⁶The median of four elements is either the second smallest or the second largest. In 2014, Ke Chen and Adrian Dumitrescu proved that if we modify MOM₄SELECT to find second-smallest elements when $k < n/2$ and second-largest elements when $k > n/2$, the resulting algorithm runs in $O(n)$ time! See their paper “Select with Groups of 3 or 4 Takes Linear Time” (WADS 2015, arXiv:1409.3600) for details.

For each b between 5 and 13, find an upper bound on the running time of $\text{MOM}_b\text{SELECT}$ of the form $T(n) \leq \alpha_b n$ for some explicit constant α_b . (For example, on page 39 we showed that $\alpha_5 \leq 16$.)

- (g) Which value of b yields the smallest constant α_b ? [Hint: This is a trick question!]

20. Prove that the variant of the Blum-Floyd-Pratt-Rivest-Tarjan SELECT algorithm shown in Figure 1.23, which uses an extra layer of small medians to choose the main pivot, runs in $O(n)$ time.

```

MOMOMSELECT( $A[1..n], k$ ):
  if  $n \leq 81$ 
    use brute force
  else
     $m \leftarrow \lceil n/3 \rceil$ 
    for  $i \leftarrow 1$  to  $m$ 
       $M[i] \leftarrow \text{MEDIANOF3}(A[3i-2..3i])$ 
     $mm \leftarrow \lceil m/3 \rceil$ 
    for  $j \leftarrow 1$  to  $mm$ 
       $Mom[j] \leftarrow \text{MEDIANOF3}(M[3j-2..3j])$ 
     $momom \leftarrow \text{MOMOMSELECT}(Mom[1..mm], \lceil mm/2 \rceil)$ 
     $r \leftarrow \text{PARTITION}(A[1..n], momom)$ 
    if  $k < r$ 
      return MOMOMSELECT( $A[1..r-1], k$ )
    else if  $k > r$ 
      return MOMOMSELECT( $A[r+1..n], k-r$ )
    else
      return  $momom$ 

```

Figure 1.23. Selection by median of moms; see problem 20).

21. (a) Suppose we are given two sorted arrays $A[1..n]$ and $B[1..n]$. Describe an algorithm to find the median element in the union of A and B in $\Theta(\log n)$ time. You can assume that the arrays contain no duplicate elements.
- (b) Suppose we are given two sorted arrays $A[1..m]$ and $B[1..n]$ and an integer k . Describe an algorithm to find the k th smallest element in $A \cup B$ in $\Theta(\log(m+n))$ time. For example, if $k = 1$, your algorithm should return the smallest element of $A \cup B$. [Hint: Use your solution to part (a).]
- ♥(c) Now suppose we are given *three* sorted arrays $A[1..n]$, $B[1..n]$, and $C[1..n]$, and an integer k . Describe an algorithm to find the k th smallest element in $A \cup B \cup C$ in $O(\log n)$ time.

- (d) Finally, suppose we are given a two dimensional array $A[1..m, 1..n]$ in which every row $A[i, \cdot]$ is sorted, and an integer k . Describe an algorithm to find the k th smallest element in A as quickly as possible. How does the running time of your algorithm depend on m ? [Hint: Solve problem 16 first.]

Arithmetic

22. In 1854, archaeologists discovered Sumerians clay tablets, carved around 2000BCE, that list the squares of integers up to 59. This discovery led some scholars to conjecture that ancient Sumerians performed multiplication by reduction to squaring, using an identity like $x \cdot y = (x^2 + y^2 - (x - y)^2)/2$. Unfortunately, those same scholars are silent on how the Sumerians supposedly squared larger numbers. Four thousand years later, we can finally rescue these Sumerian mathematicians from their lives of drudgery through the power of recursion!
- (a) Describe a variant of Karatsuba's algorithm that squares any n -digit number in $O(n^{\lg 3})$ time, by reducing to squaring three $\lceil n/2 \rceil$ -digit numbers. (Karatsuba actually did this in 1960.)
 - (b) Describe a recursive algorithm that squares any n -digit number in $O(n^{\lg_3 6})$ time, by reducing to squaring six $\lceil n/3 \rceil$ -digit numbers.
 - ♥(c) Describe a recursive algorithm that squares any n -digit number in $O(n^{\lg_3 5})$ time, by reducing to squaring only five $(n/3 + O(1))$ -digit numbers. [Hint: What is $(a + b + c)^2 + (a - b + c)^2$?]
23. (a) Describe and analyze a variant of Karatsuba's algorithm that multiplies any m -digit number and any n -digit number, for any $n \geq m$, in $O(nm^{\lg 3 - 1})$ time.
- (b) Describe an algorithm to compute the decimal representation of 2^n in $O(n^{\lg 3})$ time, using the algorithm from part (a) as a subroutine. (The standard algorithm that computes one digit at a time requires $\Theta(n^2)$ time.)
 - (c) Describe a divide-and-conquer algorithm to compute the decimal representation of an arbitrary n -bit binary number in $O(n^{\lg 3})$ time. [Hint: Watch out for an extra log factor in the running time.]
 - ♥(d) Suppose we can multiply two n -digit numbers in $O(M(n))$ time. Describe an algorithm to compute the decimal representation of an arbitrary n -bit binary number in $O(M(n) \log n)$ time. [Hint: The analysis is the hard part; use a domain transformation.]

24. Consider the following classical recursive algorithm for computing the factorial $n!$ of a non-negative integer n :

$\begin{array}{l} \text{FACTORIAL}(n): \\ \quad \text{if } n = 0 \\ \qquad \text{return } 1 \\ \quad \text{else} \\ \qquad \text{return } n \cdot \text{FACTORIAL}(n - 1) \end{array}$
--

- (a) How many multiplications does this algorithm perform?
- (b) How many bits are required to write $n!$ in binary? Express your answer in the form $\Theta(f(n))$, for some familiar function $f(n)$. [Hint: $(n/2)^{n/2} < n! < n^n$.]
- (c) Your answer to (b) should convince you that the number of multiplications is *not* a good estimate of the actual running time of FACTORIAL. We can multiply any k -digit number and any l -digit number in $O(k \cdot l)$ time using either the lattice algorithm or duplation and mediation. What is the running time of FACTORIAL if we use this multiplication algorithm as a subroutine?
- (d) The following recursive algorithm also computes the factorial function, but using a different grouping of the multiplications:

$\begin{array}{l} \text{FALLING}(n, m): \\ \quad \text{if } m = 0 \\ \qquad \text{return } 1 \\ \quad \text{else if } m = 1 \\ \qquad \text{return } n \\ \quad \text{else} \\ \qquad \text{return FALLING}(n, \lfloor m/2 \rfloor) \cdot \text{FALLING}(n - \lfloor m/2 \rfloor, \lceil m/2 \rceil) \end{array}$	$\langle\langle \text{Compute } n!/(n-m)! \rangle\rangle$
---	---

What is the running time of FALLING(n, n) if we use grade-school multiplication? [Hint: As usual, ignore the floors and ceilings.]

- (e) Describe and analyze a variant of Karatsuba's algorithm that multiplies any k -digit number and any l -digit number, for any $k \geq l$, in $O(k \cdot l^{\lg 3 - 1}) = O(k \cdot l^{0.585})$ time.
- ♥(f) What are the running times of FACTORIAL(n) and FALLING(n, n) if we use the modified Karatsuba multiplication from part (e)?
25. The **greatest common divisor** of two positive integer x and y , denoted $\text{gcd}(x, y)$, is the largest integer d such that both x/d and y/d are integers. Euclid's *Elements*, written around 300BCE, describes the following recursive algorithm to compute $\text{gcd}(x, y)$:¹⁷

¹⁷Euclid's algorithm is sometimes incorrectly described as the oldest recursive algorithm, or even the oldest *nontrivial* algorithm, even though the Egyptian duplation and mediation algorithm—which is both nontrivial and recursive—predates Euclid by at least 1500 years.

```
EuclidGCD(x, y):
  if x = y
    return x
  else if x > y
    return EuclidGCD(x - y, y)
  else
    return EuclidGCD(x, y - x)
```

- (a) Prove that EuclidGCD correctly computes $\gcd(x, y)$.¹⁸ Specifically:
- Prove that EuclidGCD(x, y) divides both x and y .
 - Prove that every divisor of x and y is a divisor of EuclidGCD(x, y).
- (b) What is the worst-case running time of EuclidGCD(x, y), as a function of x and y ? (Assume that computing $x - y$ requires $O(\log x + \log y)$ time.)
- (c) Prove that the following algorithm also computes $\gcd(x, y)$:

```
FastEuclidGCD(x, y):
  if y = 0
    return x
  else if x > y
    return FastEuclidGCD(y, x mod y)
  else
    return FastEuclidGCD(x, y mod x)
```

- (d) What is the worst-case running time of FastEuclidGCD(x, y), as a function of x and y ? (Assume that computing $x \bmod y$ takes $O(\log x \cdot \log y)$ time.)
- (e) Prove that the following algorithm also computes $\gcd(x, y)$:

```
BinaryGCD(x, y):
  if x = y
    return x
  else if x and y are both even
    return 2 · BinaryGCD(x/2, y/2)
  else if x is even
    return BinaryGCD(x/2, y)
  else if y is even
    return BinaryGCD(x, y/2)
  else if x > y
    return BinaryGCD((x - y)/2, y)
  else
    return BinaryGCD(x, (y - x)/2)
```

¹⁸Euclid did not do this. Proposition 1 in *Elements* Book VII states that if EuclidGCD(x, y) = 1, then x and y are relatively prime (that is, $\gcd(x, y) = 1$), but the proof only considers the special case $x \bmod (y \bmod (x \bmod y)) = 1$. Proposition 2 states that if x and y are *not* relatively prime, then EuclidGCD(x, y) = $\gcd(x, y)$, but the proof only considers the special cases $\gcd(x, y) = y$ and $\gcd(x, y) = y \bmod (x \bmod y)$. Finally, these two Propositions do not make a complete proof that EuclidGCD is correct. Don't be like Euclid.

- (f) What is the worst-case running time of $\text{BINARYGCD}(x, y)$, as a function of x and y ? (Assume that computing $x - y$ takes $O(\log x + \log y)$ time, and computing $z/2$ requires $O(\log z)$ time.)

Arrays

26. Suppose you are given a $2^n \times 2^n$ checkerboard with one (arbitrarily chosen) square removed. Describe and analyze an algorithm to compute a tiling of the board by without gaps or overlaps by L-shaped tiles, each composed of 3 squares. Your input is the integer n and two n -bit integers representing the row and column of the missing square. The output is a list of the positions and orientations of $(4^n - 1)/3$ tiles. Your algorithm should run in $O(4^n)$ time. [Hint: First prove that such a tiling always exists.]
27. You are a visitor at a political convention (or perhaps a faculty meeting) with n delegates; each delegate is a member of exactly one political party. It is impossible to tell which political party any delegate belongs to; in particular, you will be summarily ejected from the convention if you ask. However, you can determine whether any pair of delegates belong to the *same* party by introducing them to each other. Members of the same political party always greet each other with smiles and friendly handshakes; members of different parties always greet each other with angry stares and insults.¹⁹
- (a) Suppose more than half of the delegates belong to the same political party. Describe an efficient algorithm that identifies all members of this majority party.
- (b) Now suppose there are more than two parties, but one party has a *plurality*: more people belong to that party than to any other party. Present a practical procedure to precisely pick the people from the plurality political party as parsimoniously as possible, presuming the plurality party is composed of at least p people. Pretty please.
28. Smullyan Island has three types of inhabitants: *knight*s always speak the truth; *knave*s always lie; and *normal*s sometimes speak the truth and sometimes don't. Everyone on the island knows everyone else's name and type (knight, knave, or normal). You want to learn the type of every inhabitant.

You can ask any inhabitant to tell you the type of any other inhabitant. Specifically, if you ask "Hey X , what is Y 's type?" then X will respond as follows:

¹⁹Real-world politics is much messier than this simplified model, but this is a theory book!

- If X is a knight, then X will respond with Y 's correct type.
- If X is a knave, then X could respond with *either* of the types that Y is *not*.
- If X is a normal, then X could respond with *any* of the three types.

The inhabitants will ignore any questions not of this precise form; in particular, you may not ask an inhabitant about their own type. Asking the same inhabitant the same question multiple times always yields the same answer, so there's no point in asking any question more than once.

- Suppose you know that a strict majority of inhabitants are knights. Describe an efficient algorithm to identify the type of every inhabitant.
 - Prove that if at most half the inhabitants are knights, it is impossible to determine the type of every inhabitant.
29. Most graphics hardware includes support for a low-level operation called *blit*, or **block transfer**, which quickly copies a rectangular chunk of a pixel map (a two-dimensional array of pixel values) from one location to another. This is a two-dimensional version of the standard C library function `memcpy()`.

Suppose we want to rotate an $n \times n$ pixel map 90° clockwise. One way to do this, at least when n is a power of two, is to split the pixel map into four $n/2 \times n/2$ blocks, move each block to its proper position using a sequence of five blits, and then recursively rotate each block. (Why five? For the same reason the Tower of Hanoi puzzle needs a third peg.) Alternately, we could *first* recursively rotate the blocks and *then* blit them into place.



Figure 1.24. Two algorithms for rotating a pixel map.

- Prove that both versions of the algorithm are correct when n is a power of 2.
 - Exactly* how many blits does the algorithm perform when n is a power of 2?
 - Describe how to modify the algorithm so that it works for arbitrary n , not just powers of 2. How many blits does your modified algorithm perform?
 - What is your algorithm's running time if a $k \times k$ blit takes $O(k^2)$ time?
 - What if a $k \times k$ blit takes only $O(k)$ time?
30. An array $A[0..n-1]$ of n distinct numbers is **bitonic** if there are unique indices i and j such that $A[(i-1) \bmod n] < A[i] > A[(i+1) \bmod n]$ and

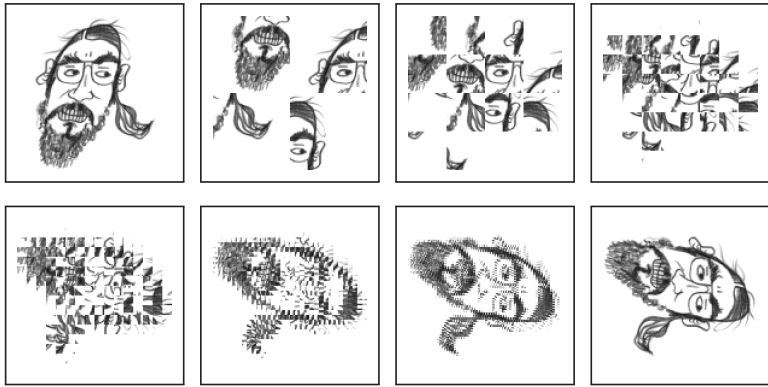


Figure 1.25. The first rotation algorithm (blit then recurse) in action. (See Image Credits at the end of the book.)

$A[(j - 1) \bmod n] > A[j] < A[(j + 1) \bmod n]$. In other words, a bitonic sequence either consists of an increasing sequence followed by a decreasing sequence, or can be circularly shifted to become so. For example,

4	6	9	8	7	5	1	2	3
3	6	9	8	7	5	1	2	4

is bitonic, but

is *not* bitonic.

Describe and analyze an algorithm to find the *smallest* element in an n -element bitonic array in $O(\log n)$ time. You may assume that the numbers in the input array are distinct.

31. Suppose we are given an array $A[1..n]$ of n distinct integers, which could be positive, negative, or zero, sorted in increasing order so that $A[1] < A[2] < \dots < A[n]$.
 - (a) Describe a fast algorithm that either computes an index i such that $A[i] = i$ or correctly reports that no such index exists.
 - (b) Suppose we know in advance that $A[1] > 0$. Describe an even faster algorithm that either computes an index i such that $A[i] = i$ or correctly reports that no such index exists. [Hint: This is **really** easy.]
32. Suppose we are given an array $A[1..n]$ with the special property that $A[1] \geq A[2]$ and $A[n-1] \leq A[n]$. We say that an element $A[x]$ is a *local minimum* if it is less than or equal to both its neighbors, or more formally, if $A[x-1] \geq A[x]$ and $A[x] \leq A[x+1]$. For example, there are six local minima in the following array:

9	7	7	2	1	3	7	5	4	7	3	3	4	8	6	9
	▲			▲				▲		▲	▲			▲	

We can obviously find a local minimum in $O(n)$ time by scanning through the array. Describe and analyze an algorithm that finds a local minimum in $O(\log n)$ time. [Hint: With the given boundary conditions, the array **must** have at least one local minimum. Why?]

33. Suppose you are given a sorted array of n distinct numbers that has been rotated k steps, for some **unknown** integer k between 1 and $n - 1$. That is, you are given an array $A[1..n]$ such that some prefix $A[1..k]$ is sorted in increasing order, the corresponding suffix $A[k+1..n]$ is sorted in increasing order, and $A[n] < A[1]$.

For example, you might be given the following 16-element array (where $k = 10$):

9	13	16	18	19	23	28	31	37	42	1	3	4	5	7	8
---	----	----	----	----	----	----	----	----	----	---	---	---	---	---	---

- (a) Describe and analyze an algorithm to compute the unknown integer k .
- (b) Describe and analyze an algorithm to determine if the given array contains a given number x .
34. At the end of the second act of the action blockbuster *Fast and Impossible XIII*^{3/4}: *The Last Guardians of Expendable Justice Reloaded*, the villainous Dr. Metaphor hypnotizes the entire Hero League/Force/Squad, arranges them in a long line at the edge of a cliff, and instructs each hero to shoot the closest taller heroes to their left and right, at a prearranged signal.

Suppose we are given the heights of all n heroes, in order from left to right, in an array $Ht[1..n]$. (To avoid salary arguments, the producers insisted that no two heroes have the same height.) Then we can compute the Left and Right targets of each hero in $O(n^2)$ time using the following brute-force algorithm.

```

WHOTARGETSWHOM( $Ht[1..n]$ ):
  for  $j \leftarrow 1$  to  $n$ 
    «Find the left target  $L[j]$  for hero  $j$ »
     $L[j] \leftarrow \text{NONE}$ 
    for  $i \leftarrow 1$  to  $j - 1$ 
      if  $Ht[i] > Ht[j]$ 
         $L[j] \leftarrow i$ 
    «Find the right target  $R[j]$  for hero  $j$ »
     $R[j] \leftarrow \text{NONE}$ 
    for  $k \leftarrow n$  down to  $j + 1$ 
      if  $Ht[k] > Ht[j]$ 
         $R[j] \leftarrow k$ 
  return  $L[1..n], R[1..n]$ 

```

- (a) Describe a divide-and-conquer algorithm that computes the output of `WHOTARGETSWHOM` in $O(n \log n)$ time.
- (b) Prove that at least $\lfloor n/2 \rfloor$ of the n heroes are targets. That is, prove that the output arrays $R[0..n-1]$ and $L[0..n-1]$ contain at least $\lfloor n/2 \rfloor$ distinct values (other than `NONE`).
- (c) Alas, Dr. Metaphor's diabolical plan is successful. At the prearranged signal, all the heroes simultaneously shoot their targets, and all targets fall over the cliff, apparently dead. Metaphor repeats his dastardly experiment over and over; after each massacre, he forces the remaining heroes to choose new targets, following the same algorithm, and then shoot their targets at the next signal. Eventually, only the shortest member of the Hero Crew/Alliance/Posse is left alive.²⁰

Describe and analyze an algorithm to compute the number of rounds before Dr. Metaphor's deadly process finally ends. For full credit, your algorithm should run in $O(n)$ time.

35. You are a contestant on the hit game show "Beat Your Neighbors!" You are presented with an $m \times n$ grid of boxes, each containing a unique number. It costs \$100 to open a box. Your goal is to find a box whose number is larger than its neighbors in the grid (above, below, left, and right). If you spend less money than any of your opponents, you win a week-long trip for two to Las Vegas and a year's supply of Rice-A-Roni™, to which you are hopelessly addicted.
- (a) Suppose $m = 1$. Describe an algorithm that finds a number that is bigger than either of its neighbors. How many boxes does your algorithm open in the worst case?
 - ♥(b) Suppose $m = n$. Describe an algorithm that finds a number that is bigger than any of its neighbors. How many boxes does your algorithm open in the worst case?
 - ♣♥(c) Prove that your solution to part (b) is optimal up to a constant factor.
36. (a) Let $n = 2^\ell - 1$ for some positive integer ℓ . Suppose someone claims to hold an unsorted array $A[1..n]$ of *distinct* ℓ -bit strings; thus, exactly one ℓ -bit string does *not* appear in A . Suppose further that the **only** way we can access A is by calling the function `FETCHBIT( $i, j$ )`, which returns the j th bit of the string $A[i]$ in $O(1)$ time. Describe an algorithm to find the missing string in A using only $O(n)$ calls to `FETCHBIT`.

²⁰In the thrilling final act, Retcon the Squirrel, the last surviving member of the Hero Team/Group/Society, saves everyone by traveling back in time and retroactively replacing the other $n - 1$ heroes with lifelike balloon sculptures. So, yeah, basically it's *Avengers: Endgame*.

- ♥(b) Now suppose $n = 2^\ell - k$ for some positive integers k and ℓ , and again we are given an array $A[1..n]$ of *distinct* ℓ -bit strings. Describe an algorithm to find the k strings that are missing from A using only $O(n \log k)$ calls to `FETCHBIT`.

Trees

37. For this problem, a *subtree* of a binary tree means any connected subgraph. A binary tree is *complete* if every internal node has two children, and every leaf has exactly the same depth. Describe and analyze a recursive algorithm to compute the *largest complete subtree* of a given binary tree. Your algorithm should return both the root and the depth of this subtree. See Figure 1.26 for an example.

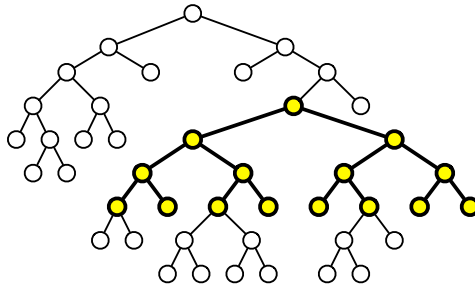


Figure 1.26. The largest complete subtree of this binary tree has depth 3.

38. Let T be a binary tree with n vertices. Deleting any vertex v splits T into at most three subtrees, containing the left child of v (if any), the right child of v (if any), and the parent of v (if any). We call v a **central** vertex if each of these smaller trees has at most $n/2$ vertices. See Figure 1.27 for an example.

Describe and analyze an algorithm to find a central vertex in an arbitrary given binary tree. *[Hint: First prove that every tree has a central vertex.]*

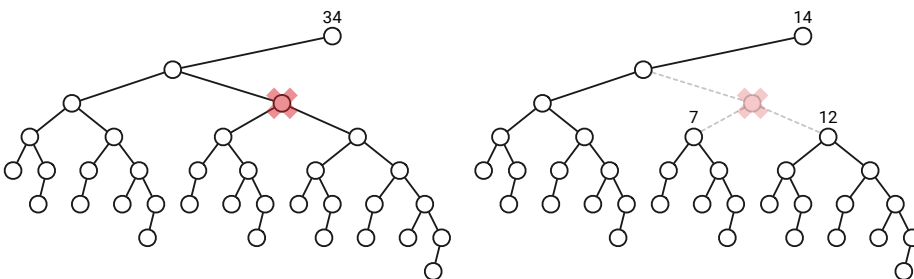


Figure 1.27. Deleting a central vertex in a 34-node binary tree, leaving subtrees with 14, 7, and 12 nodes.

39. (a) Professor George O'Jungle has a 27-node binary tree, in which every node is labeled with a unique letter of the Roman alphabet or the character &. Preorder and postorder traversals of the tree visit the nodes in the following order:

- Preorder: I Q J H L E M V O T S B R G Y Z K C A & F P N U D W X
- Postorder: H E M L J V Q S G Y R Z B T C P U D N F W & X A K O I

Draw George's binary tree.

- (b) Recall that a binary tree is *full* if every non-leaf node has exactly two children.
- i. Describe and analyze a recursive algorithm to reconstruct an arbitrary *full* binary tree, given its preorder and postorder node sequences as input.
 - ii. Prove that there is no algorithm to reconstruct an *arbitrary* binary tree from its preorder and postorder node sequences.
- (c) Describe and analyze a recursive algorithm to reconstruct an *arbitrary* binary tree, given its preorder and *inorder* node sequences as input.
- (d) Describe and analyze a recursive algorithm to reconstruct an arbitrary binary *search* tree, given only its preorder node sequence.
- ♥(e) Describe and analyze a recursive algorithm to reconstruct an arbitrary binary *search* tree, given only its preorder node sequence, *in $O(n)$ time*.

In parts (b)–(e), assume that all keys are distinct and that the input is consistent with at least one binary tree.

40. Suppose we have n points scattered inside a two-dimensional box. A **kd-tree**²¹ recursively subdivides the points as follows. If the box contains no points in its interior, we are done. Otherwise, we split the box into two smaller boxes with a *vertical* line, through a median point inside the box (*not* on its boundary), partitioning the points as evenly as possible. Then we recursively build a kd-tree for the points in each of the two smaller boxes, *after rotating them 90 degrees*. Thus, we alternate between splitting vertically and splitting horizontally at each level of recursion. The final empty boxes are called *cells*.

²¹The term “kd-tree” (pronounced “kay dee tree”) was originally an abbreviation for “ k -dimensional tree”, but modern usage ignores this etymology, in part because nobody in their right mind would *ever* use the letter k to denote dimension instead of the *obviously* superior d . Etymological consistency would require calling the data structure in this problem a “2d-tree” (or perhaps a “2-d tree”), but the standard nomenclature is now “two-dimensional kd-tree”. See also: B-tree (maybe), alpha shape, beta skeleton, epsilon net, Potomac River, Mississippi River, Lake Michigan, Lake Tahoe, Manhattan Island, La Brea Tar Pits, Sahara Desert, Mount Kilimanjaro, South Vietnam, East Timor, the Milky Way Galaxy, the City of Townsville, and self-driving automobiles.

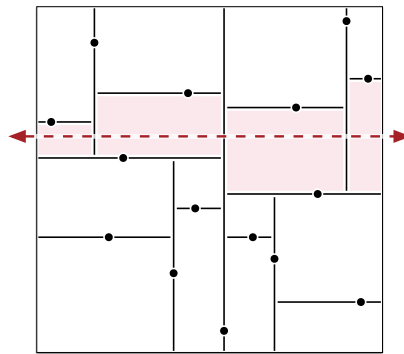


Figure 1.28. A kd-tree for 15 points. The dashed line crosses the four shaded cells.

- (a) How many cells are there, as a function of n ? Prove your answer is correct.
- (b) In the worst case, *exactly* how many cells can a horizontal line cross, as a function of n ? Prove your answer is correct. Assume that $n = 2^k - 1$ for some integer k . [Hint: There is more than one function f such that $f(16) = 4$.]
- (c) Suppose we are given n points stored in a kd-tree. Describe and analyze an algorithm that counts the number of points above a horizontal line (such as the dashed line in the figure) as quickly as possible. [Hint: Use part (b).]
- (d) Describe and analyze an efficient algorithm that counts, given a kd-tree containing n points, the number of points that lie inside a rectangle R with horizontal and vertical sides. [Hint: Use part (c).]

- ♥41. Bob Ratenbur, a new student in CS 225, is trying to write code to perform preorder, inorder, and postorder traversals of binary trees. Bob sort-of understands the basic idea behind the traversal algorithms, but whenever he actually tries to implement them, he keeps mixing up the recursive calls. Five minutes before the deadline, Bob frantically submits code with the following structure:

<pre> PREORDER(v): if v = NULL return else print label(v) ORDER(left(v)) ORDER(right(v)) </pre>	<pre> INORDER(v): if v = NULL return else ORDER(left(v)) print label(v) ORDER(right(v)) </pre>	<pre> POSTORDER(v): if v = NULL return else ORDER(left(v)) ORDER(right(v)) print label(v) </pre>
---	--	--

Each ████ in this pseudocode hides one of the prefixes PRE, IN, or POST. Moreover, each of the following function calls appears exactly once in Bob's submitted code:

PREORDER(<i>left</i> (<i>v</i>))	PREORDER(<i>right</i> (<i>v</i>))
INORDER(<i>left</i> (<i>v</i>))	INORDER(<i>right</i> (<i>v</i>))
POSTORDER(<i>left</i> (<i>v</i>))	POSTORDER(<i>right</i> (<i>v</i>))

Thus, there are precisely 36 possibilities for Bob's code. Unfortunately, Bob accidentally deleted his source code after submitting the executable, so neither you nor he knows which functions were called where.

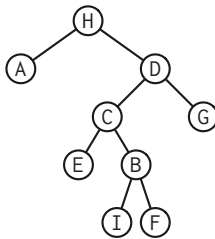
Now suppose you are given the output of Bob's traversal algorithms, executed on some **unknown** binary tree T . Bob's output has been helpfully parsed into three arrays $Pre[1..n]$, $In[1..n]$, and $Post[1..n]$. You may assume that these traversal sequences are consistent with exactly one binary tree T ; in particular, the vertex labels of the unknown tree T are distinct, and every internal node in T has exactly two children.

- Describe an algorithm to reconstruct the unknown tree T from the given traversal sequences.
- Describe an algorithm that either reconstructs Bob's code from the given traversal sequences, or correctly reports that the traversal sequences are consistent with more than one set of algorithms.

For example, given the input

$$\begin{aligned} Pre[1..n] &= [H \ A \ E \ C \ B \ I \ F \ G \ D] \\ In[1..n] &= [A \ H \ D \ C \ E \ I \ F \ B \ G] \\ Post[1..n] &= [A \ E \ I \ B \ F \ C \ D \ G \ H] \end{aligned}$$

your first algorithm should return the following tree:



and your second algorithm should reconstruct the following code:

<pre> PREORDER(<i>v</i>): if <i>v</i> = NULL return else print label(<i>v</i>) PREORDER(<i>left</i>(<i>v</i>)) POSTORDER(<i>right</i>(<i>v</i>)) </pre>

<pre> INORDER(<i>v</i>): if <i>v</i> = NULL return else POSTORDER(<i>left</i>(<i>v</i>)) print label(<i>v</i>) PREORDER(<i>right</i>(<i>v</i>)) </pre>
--

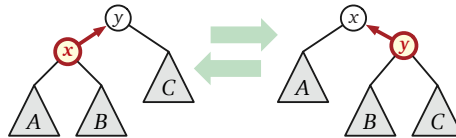
<pre> POSTORDER(<i>v</i>): if <i>v</i> = NULL return else INORDER(<i>left</i>(<i>v</i>)) INORDER(<i>right</i>(<i>v</i>)) print label(<i>v</i>) </pre>

- ♥ 42. Let T be a binary tree whose nodes store distinct numerical values. Recall that T is a **binary search tree** if and only if either (1) T is empty, or (2) T satisfies the following recursive conditions:

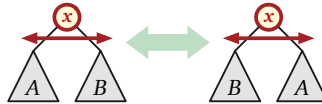
- The left subtree of T is a binary search tree.
- All values in the left subtree are smaller than the value at the root.
- The right subtree of T is a binary search tree.
- All values in the right subtree are larger than the value at the root.

Consider the following pair of operations on binary trees:

- **Rotate** an arbitrary node upward.²²



- **Swap** the left and right subtrees of an arbitrary node.



In both of these operations, some, all, or none of the subtrees A , B , and C could be empty.

- (a) Describe an algorithm to transform an *arbitrary* n -node binary tree with distinct node values into a binary search tree, using at most $O(n^2)$ rotations and swaps. Figure 1.29 shows a sequence of eight operations that transforms a five-node binary tree into a binary search tree.

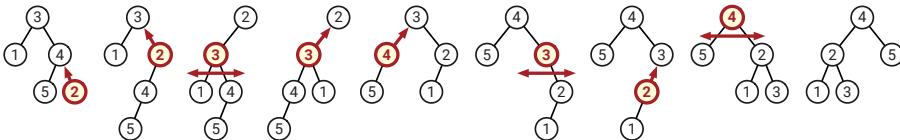


Figure 1.29. “Sorting” a binary tree: rotate 2, rotate 2, swap 3, rotate 3, rotate 4, swap 3, rotate 2, swap 4.

Your algorithm is not allowed to directly modify parent or child pointers, create new nodes, or delete old nodes; the *only* way to modify the tree is through rotations and swaps.

On the other hand, you may *compute* anything you like for free, as long as that computation does not modify the tree; the running time of your algorithm is *defined* to be the number of rotations and swaps that it performs.

- ♥(b) Describe an algorithm to transform an arbitrary n -node binary tree into a binary search tree, using at most $O(n \log n)$ rotations and swaps.

²²Rotations preserve the inorder sequence of nodes in a binary tree. Partly for this reason, rotations are used to maintain several types of balanced binary search trees, including AVL trees, red-black trees, splay trees, scapegoat trees, and treaps. See <http://algorithms.wtf> for lecture notes on most of these data structures.

- (c) Prove that any n -node *binary search* tree can be transformed into any other *binary search* tree with the same node values, using only $O(n)$ rotations (and *no* swaps).
- ♥(d) **Open problem:** Either describe an algorithm to transform an arbitrary n -node *binary tree* into a *binary search tree* using only $O(n)$ rotations and swaps, or prove that no such algorithm is possible. [*Hint: I don't think it's possible.*]

